데이터 분석과 시각화 중간고사 강의노트 (DV_01~09)

- 치트시트 — DV 01~09 한눈에 보기
  - 1. 어디에 뭐가 있나 (찾아가기)
  - 2. Plot 종류별 한 줄 요약 + 코드
  - 3. 가장 자주 쓰는 “데이터 준비” 패턴
  - 4. 꾸미기 함수 (scripting vs artist)
  - 5. 파라미터 사전 (plot() 공통)
  - 6. 개념 비교표
  - 7. Box plot / 이상치
  - 8. Matplotlib 아키텍처
  - 9. 그래프 함정 (DV 04)
  - 10. 회귀·정규화·누적
  - 11. Waffle 직접 구현 7단계
  - 12. Folium 핵심
  - 13. 시험 시간 팁
- 데이터 분석과 시각화 — 중간고사 안내 (강의노트 00)
  - 1. 시험 개요
  - 2. 허용 / 금지 한눈에 보기
  - 3. 코드파일 열람 규칙 (추가 유의사항 핵심)
  - 4. 시험 운영 규칙
  - 5. 시간 전략 (제안)
  - 6. 앞으로의 강의노트 구성 (파일 올리시는 순서대로 추가)
- DV 01 — 수업 소개 (Course Introduction)
  - 1. 교수 정보 (슬라이드 2~10)
  - 2. 학습 목표 (Learning Objectives, 슬라이드 11)
  - 3. 교재 (슬라이드 12)
  - 4. 성적 평가 (슬라이드 13)
  - 5. 퀴즈·시험 정책 (슬라이드 14)
  - 6. 출석 정책 (슬라이드 15)
  - 7. 주차별 일정 (슬라이드 16, 변경 가능)
- DV 02 — 워크스테이션 / 하드웨어 (Workstation & GPU)
  - 1. Workstation vs. Rack Server (슬라이드 2~6)
  - 2. 내 컴퓨터 사양 확인 (슬라이드 7~8)
  - 3. 컴퓨터 구성 부품 (슬라이드 9~22, 50~53)
  - 4. GPU 이름 읽는 법 (슬라이드 23~28)
  - 5. 데이터센터 GPU: B200 / GB200 / Rubin (슬라이드 29~38)
  - 6. HBM (High Bandwidth Memory) 와 GPU 데이터 흐름 (슬라이드 39~44)
  - 7. ChatGPT를 위한 GPU (슬라이드 45)
  - 8. CPU vs GPU & 시각화에 GPU가 좋은 이유 (슬라이드 46~47)
  - 9. 전원 / 케이스 (슬라이드 48~50)
- DV 03 — 데이터 시각화 개론 (Intro to Data Visualization)
  - 핵심 한 줄

    - 1. 우리 주변의 시각화 (슬라이드 2~5)
    - 2. 왜 시각화가 이해하기 쉬운가? (슬라이드 6~9)
    - 3. 사례 ① 미국인은 하루를 어떻게 보내나 (NYT) (슬라이드 10~13)
    - 4. 사례 ② Million Dollar Blocks (슬라이드 14~15)
    - 5. 의사결정을 위한 시각화 (슬라이드 16)
    - 6. 사례 ③ Ghost Map — 콜레라 (슬라이드 17~19)
    - 7. 사례 ④ 코로나 시뮬레이션 (Washington Post) (슬라이드 20~23)
    - 8. 적절한 시각화가 중요한 이유 (슬라이드 24)
    - 9. 참고 영상 (슬라이드 25)
- DV_04 — Plot 종류와 시각화 라이브러리 (Plots & Libraries 1)
    - 0. Plot 이란? (슬라이드 2~3)
    - 1. 5가지 Plot 한눈에 비교
    - 2. Line Plot (슬라이드 4~5)
    - 3. Bar Plot (슬라이드 6~8)
    - 4. Scatter Plot (슬라이드 9~11)
    - 5. Box Plot (슬라이드 12~13)
    - 6. Histogram (슬라이드 14~16)
    - 7. 주요 Python 시각화 라이브러리 (슬라이드 17~29)
    - 8. Recap (슬라이드 30)
    - 9. 한눈에: 공통 “함정” 정리 (시험 대비)
    - 10. ➕ 다음 실습(DV_05~)을 위한 대응표 (슬라이드 외 추가)
- DV_05 — Matplotlib 소개 + Line Plot 실습
- PART A. 이론 (슬라이드)
    - A-1. Matplotlib 소개 (슬라이드 2)
    - A-2. Matplotlib 아키텍처 — 3개 레이어 (슬라이드 3~8)
    - A-3. Anatomy of a figure (슬라이드 9)
    - A-4. Recap (슬라이드 10)
    - A-5. Lab 세션 안내 (슬라이드 11~16)
- PART B. Colab 실습 — Line Plot (노트북 `DV_05_code_matplotlib_line_plots.ipynb`)
    - B-0. 전체 순서 요약 (이것만 보면 흐름이 잡힘)
    - B-1. 준비 (셀 3~13)
    - B-2. Matplotlib 불러오기 (셀 14~21)
    - B-3. pandas로 plot 그리기 (셀 22)
    - B-4. 사례: 아이티 지진 Line plot (셀 24~34)
    - B-5. 여러 나라 비교: India vs China (셀 35~48)
    - B-6. 이민 상위 5개국 추세 (셀 49~52)
- PART C. Line Plot 정리 — “plot별 사용 변수” 표
    - 꾸미기 함수 모음
    - 자주 틀리는 포인트 (시험 대비)
- DV_06 — Area Plot · Histogram · Bar Chart (Matplotlib + pandas)
- PART A. 이론 (슬라이드)
    - A-1. Area Plot (슬라이드 2, 8~9)
    - A-2. Histogram (슬라이드 10)
    - A-3. Bar Chart (슬라이드 15~18)
    - A-4. Histogram vs Bar (슬라이드 19~20) ⭐ 시험 단골 비교

    - A-5. Recap (슬라이드 21)
- PART B. Colab 실습 (노트북 `DV_06_Area_Hist_Bar.ipynb`)
    - B-0. 전체 순서 요약
    - B-1. 공통 준비 (셀 1~13)
    - B-2. Area Plot
    - ★ B-3. 두 가지 plotting 스타일 (셀 23~27) — 개념 이해 필수
    - B-4. Histogram
    - B-5. Bar Chart
    - B-6. Histogram vs Bar 직접 비교 (셀 74~82)
- PART C. 정리표
    - C-1. plot별 사용 변수·함수
    - C-2. 파라미터 사전
    - C-3. 시험 대비 포인트
- DV 07 — Pie · Box · Scatter · Bubble Plot
- PART A. 이론 (슬라이드)
    - A-1. Pie Chart (슬라이드 2~5)
    - A-2. Box Plot (슬라이드 6~7)
    - A-3. Scatter Plot (슬라이드 8~11)
    - A-4. Recap (슬라이드 12)
- PART B. Colab 실습 (노트북 `DV_07_code_pie_box_scatter_bubble_1.ipynb`)
    - B-0. 전체 순서
    - B-1. 준비 (셀 2~15)
    - B-2. Pie Chart
    - B-3. Box Plot
    - B-4. Scatter Plot
    - B-5. Bubble Plot
- PART C. 정리표
    - C-1. plot별 사용 변수·데이터
    - C-2. 파라미터 사전 (DV 07 신규)
    - C-3. 시험 대비 포인트
- DV 08 — Waffle Chart · Word Cloud · Seaborn(Categorical / Regression)
- PART A. 이론 (슬라이드)
    - A-1. Waffle Chart (슬라이드 2~4)
    - A-2. Word Cloud (슬라이드 5~6)
    - A-3. Seaborn (슬라이드 7)
    - A-4. Regression Plot (슬라이드 8~11)
    - A-5. Categorical Plots (슬라이드 12~13)
    - A-6. Recap (슬라이드 14)
- PART B. Colab 실습 (노트북 `DV_08_code_waffle_wordcloud_regression.ipynb`)
    - B-0. 전체 순서
    - B-1. 준비 (셀 2~10)
    - B-2. Waffle Chart
    - B-3. Word Cloud
    - B-4. Seaborn
- PART C. 정리표

    - C-1. plot별 사용 변수·함수
    - C-2. 시험 대비 포인트
- DV 09 — Folium으로 지도 만들기 (Maps · Markers · Choropleth)
- PART A. 이론 (슬라이드)
    - A-1. Folium이란? (슬라이드 2)
    - A-2. 지도 만들기 (슬라이드 3~6)
    - A-3. 마커가 있는 지도 (슬라이드 7~13)
    - A-4. Choropleth Map (슬라이드 14~17)
    - A-5. Recap 요약 (슬라이드 18)
- PART B. Colab 실습 (노트북 `DV_09_Creating_map_by_folium_1.ipynb`)
    - B-0. 전체 순서
    - B-1. 준비 (셀 5~8)
    - B-2. 기본 지도와 확대 수준 (셀 11~21)
    - B-3. 지도 스타일(tiles) (셀 22~34)
    - B-4. 마커가 있는 지도: 샌프란시스코 경찰 사건 데이터
    - B-5. Choropleth Map: 세계 이민 지도
- PART C. 정리표
    - C-1. Folium 객체·함수 정리
    - C-2. plot별 사용 변수(데이터)
    - C-3. 시험 대비 포인트

치트시트 — DV_01~09 한눈에 보기

> 오픈북 시험에서 먼저 펼쳐볼 한 장 요약입니다. 자세한 설명은 각 강의 노트(DV_0X)에서 찾으세요. 표기: 노트 번호는 `[05]` = DV_05 노트

1. 어디에 뭐가 있나 (찾아가기)

찾는 내용	노트
시험 규칙, 허용/금지	00
수업 소개, 성적 비율	01
GPU 이름(RTX 5070), HBM, CPU vs GPU	02
시각화가 효과적인 이유, 사례(NYT, John Snow, 코로나)	03
Line/Bar/Scatter/Box/Histogram 개념과 함정, 라이브러리 비교	04
Matplotlib 3개 레이어, Anatomy, Line plot 실습	05
Area / Histogram / Bar 실습, scripting vs artist layer	06
Pie / Box / Scatter / Bubble 실습, subplot, 회귀선	07
Waffle / Word cloud / Seaborn(countplot, barplot, regplot)	08
Folium 지도 / 마커 / Choropleth	09

2. Plot 종류별 한 줄 요약 + 코드

Plot	언제	데이터 모양	핵심 코드
Line	시간에 따른 추세	index=x(연도), 열=선	`df.plot(kind='line')`
Area	누적 합/비율 추세	line과 동일 (기본 stacked)	`df.plot(kind='area', stacked=, alpha=)`
Histogram	연속형 1변수의 분포	열 하나 (Series)	`df['2013'].plot(kind='hist', bins=, xticks=bin_edges)`
Bar / Barh	범주 간 값 비교	index=범주, 값 하나	`s.plot(kind='bar')` / `'barh'`
Pie	전체 중 비율	`groupby().sum()` 의 한 열	`df['Total'].plot(kind='pie', autopct='%1.1f%%')`
Box	분포(5수 요약)와 이상치	열 = 그룹 (`transpose()`)	`df.plot(kind='box', vert=)`
Scatter	두 연속변수 관계	x, y 열 이름 지정	`df.plot(kind='scatter', x='year', y='total')`
Bubble	scatter + 3번째 변수(크기)	정규화한 가중치	`scatter(..., s=norm*2000+10, ax=ax0)`

Plot	언제	데이터 모양	핵심 코드
Waffle	범주 비율을 타일로	범주별 값	`plt.figure(FigureClass=Waffle, rows, columns, values)`
Word cloud	텍스트 단어 빈도	문자열	`WordCloud(stopwords, mask).generate(text)`
countplot	범주별 개수	범주형 열 (y 없음)	`sns.countplot(x=, data=)`
barplot	범주별 평균 + CI	범주 x, 수치 y	`sns.barplot(x=, y=, data=)`
regplot	scatter + 회귀선 + CI	수치 x, 수치 y	`sns.regplot(x=, y=, data=, ci=95)`
Folium 지도	위치/지역 시각화	위도·경도 / GeoJSON	`folium.Map(location=[위도,경도], zoom_start=)`

3. 가장 자주 쓰는 “데이터 준비” 패턴

```
        df_can.set_index('Country', inplace=True)                # 국가명을 index로
        years = list(map(str, range(1980, 2014)))                # 연도 열 이름이 문자열이라 문자열 리스트
        df_can.loc['Haiti', years]                               # 한 나라 → Series
        df_can.loc[['India','China'], years]                     # 여러 나라 → DataFrame (리스트로 전달)
        df.transpose()                                           # 행↔열 (index=연도, 열=국가)
        df.index = df.index.map(int)                             # 연도 index를 정수로
        df_can.sort_values(['Total'], ascending=False, axis=0, inplace=True)   # Total 내림차순
        df_can.head(5)  /  df_can.tail(5)                        # 상위 5 / 하위 5
        df_can[years].sum(axis=0)    # 열별(연도별) 합     |  .sum(axis=1)  # 행별(국가별) 합
        df_can.groupby('Continent').sum()                        # Split-Apply-Combine
        df_tot.reset_index(inplace=True); df_tot.columns = ['year','total']   # index→열, 이름 변경
        (x - x.min()) / (x.max() - x.min())                      # min-max 정규화 [0,1]
```

4. 꾸미기 함수 (scripting vs artist)

하고 싶은 것	Scripting (**`plt.`**)	Artist (**`ax.`**)
제목	`plt.title()`	`ax.set_title()`
x축 이름	`plt.xlabel()`	`ax.set_xlabel()`
y축 이름	`plt.ylabel()`	`ax.set_ylabel()`
텍스트	`plt.text(x, y, '글')`	—
화살표/주석	`plt.annotate(s, xy, xytext, arrowprops=)`	—
범례	`plt.legend(labels=, loc=)`	`ax.legend()`
표시	`plt.show()` (꾸미기는 show 앞에)	

> `ax = df.plot(...)` 으로 Axes를 받으면 artist 방식, 겹쳐 그리기는 `plot(..., ax=ax0)`. Subplot: `fig = plt.figure(); ax0 = fig.add_subplot(1, 2, 1)` → `df.plot(..., ax=ax0)`.

5. 파라미터 사전 (plot() 공통)

파라미터	의미	쓰는 plot
`kind`	plot 종류	전부
`figsize=(w,h)`	그림 크기	전부
`alpha`	투명도 0~1	area, hist, scatter
`stacked`	쌓기	area(기본 True), hist
`bins`	구간 수	hist
`xticks=bin_edges`	눈금을 bin 경계에	hist
`xlim=(min,max)`	x 범위	hist, scatter
`color`, `edgecolor`	색	hist, bar, box, scatter
`rot`	눈금 글자 회전	bar
`vert=False`	가로 방향	box
`s`	점(버블) 크기	scatter
`autopct`, `startangle`, `shadow`, `labels=None`, `pctdistance`, `explode`, `colors`	pie 꾸미기	pie
`ax`	그릴 Axes	전부

6. 개념 비교표

Histogram vs Bar

	Histogram	Bar
데이터	연속형	범주형
간격	없음	있음
순서	유지	변경 가능
목적	분포	범주 비교

Matplotlib vs Seaborn

	Matplotlib	Seaborn
표현력	완전 커스터마이징	통계 표현 기본 제공
디자인	투박, 직접 꾸밈	깔끔
통계	별도 라이브러리	boxplot/violin/regression 자동
코드	길다	짧다

	Matplotlib	Seaborn
데이터	numpy 배열	pandas DataFrame (열 이름)

stacked vs unstacked area

- stacked(기본): 쌓아서 전체 합과 기여도 / unstacked: `stacked=False`, 겹침 (기본 alpha 0.5)

countplot vs barplot

- countplot: y 없음, 개수 / barplot: y 지정, 평균 + 신뢰구간

7. Box plot / 이상치

- 5수: min, Q1, median, Q3, max / IQR = Q3 − Q1
- 이상치: > Q3 + 1.5×IQR 또는 < Q1 − 1.5×IQR
- 한 나라만 DataFrame으로: `.loc[['Japan'], years]` (대괄호 두 겹) → `describe()` 가능

8. Matplotlib 아키텍처

- Backend(FigureCanvas, Renderer, Event) → Artist(Primitive: Line2D, Text, Rectangle / Composite: Axis, Tick, Axes, Figure) → Scripting(pyplot)
- 만든 사람 John Hunter, EEG/ECoG 시각화에서 시작

9. 그래프 함정 (DV_04)

Plot	함정	해결
Line / Bar	y축 시작값이 0이 아님 → 변화 과장	y축 0부터
Scatter	이상치	포함·제외 판단
Box	이상치 무시	정보 활용
Histogram	bin 수	too few / optimal / too many

10. 회귀·정규화·누적

- 회귀선: `fit = np.polyfit(x, y, deg=1)` → `[기울기, 절편]` → `plt.plot(x, fit[0]*x + fit[1])`
- CI: 신뢰수준 높을수록(99%) 음영 폭 ↑
- `plt.text(x, y, ...)`: x축이 int면 값, str(막대)면 순번

11. Waffle 직접 구현 7단계

① 비율 ② 전체 크기(width×height) ③ 범주별 타일 수 ④ 행렬 채우기 ⑤ `plt.matshow` ⑥ 격자(minor ticks) ⑦ 범례 (`mpatches.Patch`)

12. Folium 핵심

```
m = folium.Map(location=[위도, 경도], zoom_start=4, tiles='Cartodb positron')
folium.Marker([lat, lng], popup='라벨').add_to(m)                    # 마커
plugins.MarkerCluster().add_to(m)  →  Marker(...).add_to(cluster)    # 군집
folium.Choropleth(geo_data=geojson, data=df, columns=['Country','Total'],
                  key_on='feature.properties.name', fill_color='YlOrRd',
                  legend_name='...').add_to(m)                       # 구역 색칠
```

- 데이터: X = 경도, Y = 위도 → 지도에는 `[Y, X]`
- zoom_start 클수록 확대 / popup = 클릭 시 라벨 / GeoJSON 필수 / key_on 대소문자 구분

13. 시험 시간 팁

- 50분 자동 종료·자동 제출 → 모르는 문제도 먼저 답 선택
- 코드 찾기는 이 치트시트의 [찾아가기]부터

데이터 분석과 시각화 — 중간고사 안내 (강의노트 00)

> 출처: Midterm Exam Notice (영문/국문 슬라이드) + 추가 유의사항(코드파일 열람 규칙)

1. 시험 개요

항목	내용
날짜	10월 19일 (월)
시간	11:00 ~ 12:00 (실제 시험 시간 50분)
장소	B345호
형식	객관식 + 단답형 + 해설형(서술형)
유형	오픈북 (아래 허용/금지 범위 엄수)
범위	~ DV_09 (DV_01 ~ DV_09)

2. 허용 / 금지 한눈에 보기

✅ 허용 (좌석에 비치된 **데스크톱만** 사용)

- 강의 노트, 코드(변환된 파일), 개인 메모 열람
- 인터넷 검색
- 번역기 (구글 번역, 파파고)
- 종이에 인쇄한 자료 (추가 유의사항에 명시)

❌ 금지

- 개인 기기: 태블릿, 노트북, 휴대폰
- 생성형 AI 전부 (답/코드 요청, Colab 내 Gemini 포함)
- 코드 작성/실행 플랫폼 전부: Colab 자체, PyCharm, Visual Studio 등
- .ipynb 파일을 Colab이나 IDE로 여는 것 (변환 목적이어도 불가)
- 모든 메신저(카카오톡, 메신저 등)로 답안 공유
- 시험 중 퇴실 후 재입실

> ⚠️ 위반 시 자동 F 학점

3. 코드파일 열람 규칙 (추가 유의사항 핵심)

시험 중에 코드를 보고 싶다면 시험 전에 미리 준비:

1. `.ipynb` → HTML 또는 PDF로 변환 (Colab: 파일 → 다운로드 / 인쇄 → PDF 저장, 또는 `jupyter nbconvert --to html`)
2. 변환 파일을 시험용 컴퓨터에서 열 수 있게 준비하거나 종이로 인쇄
3. 시험장에서는 변환된 HTML/PDF만 열기 — .ipynb를 Colab/IDE로 여는 것은 변환 목적이어도 절대 금지

준비 체크리스트

- ☐ DV_01 ~ DV_09 실습 노트북 전부 HTML/PDF 변환
- ☐ 변환 파일을 시험 컴퓨터에서 접근 가능한 방법 확인 (본인이 가져올 수 있는 방식은 감독자에게 사전 확인 — 개인 기기 반입 금지이므로 USB/클라우드 가능 여부 불명)
- ☐ 강의노트(이 노트) PDF/종이 인쇄본 준비
- ☐ PDF는 Ctrl+F 검색이 되는지 확인 (이미지 캡처본은 검색 불가)
- ☐ 노트북 목차(plot 종류별 함수·변수 요약표) 1장짜리 치트시트 인쇄

4. 시험 운영 규칙

- 50분 00초에 프로그램 자동 종료 → 답안 자동 제출 (Submit 버튼 안 눌러도 제출됨), 재오픈 불가
- 모르는 문제도 일단 답을 찍어두고 나중에 재검토
- 시험 전 화장실 이용 (중간 퇴실·재입실 불가)
- 끝나면 시험지 제출 후 조기 퇴실 가능
- 시험 중 돌아오는 출석부에 이름 작성 + 서명 (국문 슬라이드에는 "시험감독 보고서에 서명"으로 표기 — 현장 안내에 따를 것)

5. 시간 전략 (제안)

- 50분 / 문항 수 미확인 → 객관식·단답 먼저(빠르게), 해설형은 마지막에 시간 배분
- 코드 찾아보는 시간이 가장 큰 함정: 어느 노트북의 어느 plot인지 목차로 즉시 찾도록 정리해 두기
- 오픈북이라도 검색에 시간 쓰면 부족 → 암기 + 치트시트 우선

6. 앞으로의 강의노트 구성 (파일 올리시는 순서대로 추가)

각 DV_XX 강의마다 `notes/DV_XX_제목.md` 형태로 아래 틀을 사용:

1. 핵심 개념 (영어 용어 → 한국어, 정의)
2. Colab 실습 순서 (import → 데이터 로드 → 전처리 → plot → 꾸미기)
3. Plot별 사용 변수 표 (plot 종류 / 함수 / x·y·hue 등에 들어가는 변수 유형 / 주요 옵션)
4. 암기 포인트 & 시험 예상 질문

범위가 DV_09까지이므로 총 9개 강의 + 최종 치트시트 1장으로 마무리 예정.

DV_01 — 수업 소개 (Course Introduction)

> 표기: 📘 = 슬라이드 내용 / 💡 = 이해용 주석 / ➕ = 슬라이드에 없는 추가 설명 DV_01은 실습 자료 없음. 시험 출제 가능성은 낮지만 내용은 빠짐없이 정리.

1. 교수 정보 (슬라이드 2~10)

- 📘 신유진 (Youjin Shin), 조교수 (데이터사이언스학과, 인공지능학과)
- 📘 박사: 뉴욕주립대 스토니브룩(SUNY Stony Brook) 컴퓨터공학
- 📘 연구 분야: Medical AI, 이상 탐지·예측(Anomaly Detection & Prediction), 시계열 분석(Time-series Analysis)
- 📘 이메일 yj.shinn@catholic.ac.kr / 연구실 T405 / 02-2164-5524 / 홈페이지 https://sites.google.com/view/adslab0
- 📘 슬라이드 3~10: 연구 소개 (Medical AI, 이상 탐지·예측, 시계열 분석, 기타: Machine Unlearning, Authentication, Deepfake Detection, 논문 KDD 2024 / KDD 2026)

2. 학습 목표 (Learning Objectives, 슬라이드 11)

📘 이 과정은 데이터 시각화의 기본 원리를 소개하고, Python으로 의미 있고 통찰력 있으며 효과적인 시각화를 만드는 것에 초점. Matplotlib, Seaborn 라이브러리를 배우고 실제 데이터셋에 적용. 시각화의 미적 측면 + 기능적 측면 모두 강조.

5가지 목표:

1. 데이터 분석에서 시각화의 원리와 중요성에 대한 포괄적 이해
2. 다양한 유형의 시각화를 생성·맞춤 설정하는 도구와 기법에 능숙해지기
3. 시각화를 위한 데이터를 정확성·명확성을 보장하며 준비·관리하기
4. 시각적 표현을 통해 얻은 통찰을 해석·전달하는 기술 향상
5. 다양한 데이터 유형과 대상(청중)에 맞는 시각화를 설계·평가한 경험 쌓기

3. 교재 (슬라이드 12)

교재	노트
Python for Data Analysis — Wes McKinney (O'Reilly)	약간 기본
Python Data Science Handbook — Jake VanderPlas (O'Reilly)	약간 중고급
Machine Learning with PyTorch and Scikit-Learn — Sebastian Raschka	『머신 러닝 교과서 with 파이토치』. 핵심 라이브러리(SciPy, NumPy, scikit-learn, Matplotlib, Pandas)로 ML, PyTorch로 딥러닝 설명

4. 성적 평가 (슬라이드 13)

항목	비율
중간고사	30%
기말 프로젝트(또는 기말고사)	30%
수업 내 과제(또는 퀴즈)	30%
출석	10%

절대평가: A+ 95~100 / A0 90~95 / B+ 85~90 / B0 80~85 / C+ 75~80 / C0 70~75 / D+ 65~70 / D0 60~65 / F 60 미만

5. 퀴즈·시험 정책 (슬라이드 14)

- 퀴즈: 간단한 10문항 — yes/no, 객관식, 단답형, 한 문장 설명
- 수업 내 과제(In-class Assignment): Python 코드로 시각화 실습 문제 풀기 / 컴퓨터로 코드를 작성해 plot·figure 생성 / 교실을 나가기 전에 제출 완료

6. 출석 정책 (슬라이드 15)

- 수업 시작 후 5분 이내 출석 / 30분 이내 지각 (지각 3회 = 결석 1회) / 30분 이후 결석
- 기본은 사이버캠퍼스 출석. 단, 온라인 체크인만 하고 호명 시 교실에 없으면 결석 3회로 처리

7. 주차별 일정 (슬라이드 16, 변경 가능)

주차	내용
W1	Course Introduction
W2	Introduction to Data Visualization
W3	Visualization Library (Matplotlib)
W4~W5	Visualization Library (Seaborn)
W6~W7	Statistical Analysis
W8	Midterm Period
W9	Happiness Index Data Analysis
W10	Geographic Information Analysis
W11	Hospital No-Show Patient Analysis
W12	Stock Prediction
W13	Movie Review Visualization
W14	Health Data Analysis
W15	Make-up Classes
W16	Final Exam Period

💡 중간고사 범위 “~DV_09”는 W1~W7 내용(DV_01~09: 소개 → 시각화 개론 → Matplotlib → Seaborn → 통계분석)으로 보임. DV 번호와 주차가 1:1은 아닐 수 있음(DV_02는 하드웨어 강의).

DV_02 — 워크스테이션 / 하드웨어 (Workstation & GPU)

> 표기: 📘 = 슬라이드 / 💡 = 이해용 주석 / ➕ = 추가 설명 파일명은 “HW”(hardware). 코딩이 아니라 컴퓨터 부품과 GPU 개념 강의. 이해 위주.

1. Workstation vs. Rack Server (슬라이드 2~6)

- 📘 두 가지 형태를 사진으로 비교. Rack Server(서버실 랙에 장착) vs Workstation(개인용 고성능 데스크톱형)
- 📘 Workstation 구성 견적 예시: 2023년 구성, 2025년 구성 약 200만원
- 💡 이 수업의 시각화·딥러닝 작업용 개인 컴퓨터가 workstation.

2. 내 컴퓨터 사양 확인 (슬라이드 7~8)

- 📘 ‘시스템 정보’ 앱 → 윈도우 + R → `msinfo32` 입력

3. 컴퓨터 구성 부품 (슬라이드 9~22, 50~53)

(슬라이드 10: 부품 배치도 — CPU, CPU Cooler, RAM(heat spreader), SSD(SATA / M.2), Power, GPU, Thermal compound)

부품	설명
Mainboard (= mother board)	모든 부품이 꽂히는 판. CPU전원·메인보드 전원 커넥터, SATA(SSD/HDD) 포트, M.2 슬롯, RAM 슬롯, GPU 슬롯
메인보드 크기(폼 팩터)	확장 ATX: 최대(12×13 또는 12×10.1인치), 보통 RAM 슬롯 8개(최대 128GB) / ATX: 12×9.6인치, RAM 슬롯 4개 / microATX: 9.6×9.6인치, 최대 4개 / 미니 ITX: 가장 작음(6.7×6.7인치), RAM 슬롯 2개
CPU	AMD Ryzen (5, 7, 9, Threadripper) vs Intel Core (i5, i7, i9, Xeon)
CPU Cooler	CPU 열 식힘. CPU와 쿨러 사이에 Thermal compound(써멀 컴파운드)
RAM (Memory)	작업 중 데이터를 임시로 올려두는 메모리 (방열판 heat spreader)
Storage – SSD	SATA 방식 / M.2 방식
Storage – HDD	대용량 저장
GPU	그래픽카드. 아래 상세
Power	전원 공급 장치(파워)
Case	케이스

- 📘 조립 참고: Intel “how to build a gaming PC” 링크 (슬라이드 51~53: 조립 과정 사진)

4. GPU 이름 읽는 법 (슬라이드 23~28)

📘 GeForce RTX 5070 = 세대(50) + 성능(70)

- 앞 두 자리 = Generation (세대): 10, 20, 30, 40, 50 …
- 뒤 두 자리 = Performance (성능): 50~90
- 📘 세대가 높다고 항상 성능이 좋은 것은 아니다!
- 📘 딥러닝에는 세대가 낮아도 성능 70 이상 선택 권장
- 💡 예: RTX 5060(최신 세대, 성능 60) < RTX 4070(이전 세대, 성능 70) 일 수 있음 → 딥러닝은 70 이상이 유리.
- 📘 슬라이드 25~27: GPU 성능 벤치마크 막대그래프, Nvidia GPU 라인업(소비자용 / Enterprise & Data Center GPU)

5. 데이터센터 GPU: B200 / GB200 / Rubin (슬라이드 29~38)

이름	의미
B200	Blackwell 아키텍처의 GPU (B = Blackwell)
GB200	Grace CPU 1개 + Blackwell GPU 2개를 결합한 Superchip (G = Grace CPU) → “GPU에 CPU까지 붙여 하나의 AI용 슈퍼칩”
DGX B200	B200 GPU 8개 탑재 완제품 서버
GB200 NVL72	Grace CPU 36개 + Blackwell GPU 72개를 한 랙에 구성
Rubin GPU	B200의 다음 세대 GPU
Vera Rubin Superchip	Vera CPU 1개 + Rubin GPU 2개
Vera Rubin NVL72	Vera CPU 36개 + Rubin GPU 72개, 한 랙

- 📘 정리: GPU 차원 B200 → Rubin GPU / 시스템 차원 GB200 NVL72 → Vera Rubin NVL72
- 💡 Blackwell 세대 ↔ Rubin 세대, Grace ↔ Vera (CPU 이름)로 짝을 맞춰 기억.

가격 (슬라이드 38)

제품	가격 (2025.07 기준, “지금은 더 오름”)
B200 GPU 단품	약 3~4만 달러 (4,000만~5,500만원)
DGX B200 (GPU 8개)	약 51만 달러~ (7억원 이상)
GB200 NVL72	약 300만~350만 달러 (42억~49억원)
Vera Rubin NVL72 (2026.07 가격)	업계 추정 약 800만~900만 달러 (110억~130억원)

6. HBM (High Bandwidth Memory) 와 GPU 데이터 흐름 (슬라이드 39~44)

- 📘 HBM = High Bandwidth Memory, GPU 바로 옆에 붙은 초고대역폭 메모리 (슬라이드 41~42: GPU 칩 주변에 HBM이 붙은 그림)

GPU 연산 시 데이터 흐름 (LLM 학습 예)

1. SSD → CPU RAM: 학습 데이터가 SSD에서 시스템 RAM으로 로딩
2. CPU RAM → HBM: PyTorch의 `tensor.to("cuda")` 가 데이터를 GPU용 HBM으로 복사

3. HBM → GPU 연산 유닛: Tensor Core / CUDA Core가 연산에 필요한 데이터를 HBM에서 읽음

- 📘 “4.8 TB/s” 같은 HBM bandwidth는 주로 3단계(HBM → GPU 연산 유닛) 의 전송 속도.

단순화 예: Y = XW

- X(input/activation) → HBM, W(model weights) → HBM → Tensor Core가 XW 계산 → 결과 Y → 다시 HBM. 이것이 엄청난 속도로 반복.
- 📘 그래서 LLM에서는 계산 성능(TFLOPS/PFLOPS)만큼 HBM bandwidth가 중요.
- 📘 Tensor Core가 아무리 빨라도 HBM이 데이터를 제때 못 주면 Tensor Core가 논다 → memory bottleneck(메모리 병목).

7. ChatGPT를 위한 GPU (슬라이드 45)

- 📘 NVIDIA H100 / H200: OpenAI 대규모 AI 학습 인프라에 사용. GPT-5.2는 H100, H200, GB200-NVL72 기반 인프라로 개발됐다고 OpenAI가 공식 발표 → GPT-5 세대의 핵심 학습 인프라.
- 📘 GB200 (Blackwell): 최근 OpenAI 프런티어 모델 학습이 Hopper(H100/H200) → Blackwell(GB200) 로 이동. Stargate 미국 Abilene 데이터센터는 GB200 기반, GPT-5.5가 여기서 학습됐다고 공식 발표.
- 📘 Vera Rubin: OpenAI–NVIDIA, 최소 10 GW 규모 NVIDIA 시스템 구축 파트너십. 첫 1 GW는 2026년 하반기 Vera Rubin 플랫폼 기반 예정(장기적으로 수백만 개 GPU 규모).
- 💡 세대 순서: Hopper(H100/H200) → Blackwell(B200/GB200) → Rubin(Vera Rubin).

8. CPU vs GPU & 시각화에 GPU가 좋은 이유 (슬라이드 46~47)

- 📘 CPU: 개수는 적지만 똑똑한(복잡한 작업을 잘하는) 두뇌
- 📘 GPU: 하나하나는 덜 똑똑하지만 엄청 많다
- 📘 시각화에 GPU가 유리한 이유: 이미지는 픽셀(pixel)들로 구성됨 (원 그림을 픽셀 격자로 확대한 슬라이드)
- 💡 수많은 픽셀에 같은 단순 계산을 동시에(병렬로) 해야 하므로, 단순 연산 코어가 많은 GPU가 적합.

9. 전원 / 케이스 (슬라이드 48~50)

- Power(파워), Case, HDD 사진 위주 슬라이드 (설명 텍스트 없음).

DV_03 — 데이터 시각화 개론 (Intro to Data Visualization)

> 표기: 📘 = 슬라이드 / 💡 = 이해용 주석 / ➕ = 추가 설명

핵심 한 줄

시각화가 효과적인 이유 = 사람은 시각 정보를 가장 빠르게 처리 + 큰 데이터를 압축 + 숨은 패턴이 드러남 → 의사결정에 도움.

1. 우리 주변의 시각화 (슬라이드 2~5)

- 📘 지도(내비게이션 경로), 태풍 진로 지도 등 일상에 시각화가 이미 깊이 들어와 있음.

2. 왜 시각화가 이해하기 쉬운가? (슬라이드 6~9)

감각별 정보 처리량 (슬라이드 7~8)

- 📘 덴마크 물리학자 Tor Nørretranders: 사람이 시간당 처리하는 정보량은 감각 입력마다 다름. 감각 처리를 컴퓨터 시스템에 비유 (David McCandless의 TED Talk 도표).

감각	비유
시각 (sight)	약 1,250 MB/s ≈ 컴퓨터 네트워크(network)
촉각 (touch)	USB 연결
청각·후각 (hearing, smell)	하드디스크(hard disk)

- 📘 한눈에 봐도 시각이 다른 감각보다 시간당 훨씬 많은 정보를 처리.

텍스트 vs 그래픽 (슬라이드 9)

- 📘 3M 연구: 그래픽은 텍스트보다 60,000배 빨리 처리됨
- 📘 Dan Zarrella (HubSpot): 이미지가 있는 트윗은 없는 트윗보다 95% 더 많이 리트윗됨

3. 사례 ① 미국인은 하루를 어떻게 보내나 (NYT) (슬라이드 10~13)

- 📘 2008 American Time Use Survey (Bureau of Labor Statistics): 수면, 집안일 등 활동 범주별 하루 시간 → 표만 보면 특별한 통찰 찾기 어려움
- 📘 New York Times의 시각화:
  - 활동(식사, 일, 집안일, 이동, TV·영화, 사교 …)마다 고유한 색
  - 활동에 쓴 시간 = 면적 크기
  - 우측 상단에서 직업·성별·인종별로 세분화 → 클릭하면 집단별 하루 확인 (인터랙티브)

- 📘 발견: 취업자 vs 실업자 — TV 시청 시간은 비슷(진한 갈색 영역) / 취업자는 일 시간 훨씬 많고 교육 시간 적음 / 취업자 식사 시간이 더 규칙적
- 📘 결론: 수학·통계 배경이 없어도 표로는 알기 어려운 패턴·통찰 발견 가능.

4. 사례 ② Million Dollar Blocks (슬라이드 14~15)

- 📘 Columbia University Center for Spatial Research (Laura Kurgan): 미국 수감 비용과 수감자 거주지 연구
- 📘 거주지별 수감 비용 지도 — 수감 비용이 클수록 더 밝은 빨강
- 📘 발견: 비용이 특정 지역에 심하게 집중. 가장 밝은 곳은 뉴욕 브루클린의 11개 블록으로 2003년 한 해에만 150억 원 이상(슬라이드 표기). 이 지역은 교육 등 공공 자원이 부족.
- 📘 시각화가 지역사회의 중요한 문제를 식별하도록 도움.

5. 의사결정을 위한 시각화 (슬라이드 16)

시각화가 의사결정을 돕는 이유 3가지:

1. 대량의 데이터를 압축 — 수천 행을 하나의 그림으로 → 큰 그림을 한눈에
2. 이해하기 쉬움 — 과학·수학·통계 배경 없이도 이해
3. 뇌는 시각 정보 처리에 뛰어남 — 숨은 통찰·패턴을 발견, 복잡한 데이터를 직관적으로 이해해 의사결정

6. 사례 ③ Ghost Map — 콜레라 (슬라이드 17~19)

- 📘 1854년 런던 콜레라: 10일 만에 500명 이상 사망. 당시 정부는 악취(냄새)가 원인이라 믿고 시민들에게 하수를 템스강에 버리라 지시 → 강물이 식수라 오히려 악화.
- 📘 의사 John Snow: 오래전부터 오염된 물로 전염된다고 주장했으나 무시당함. 발병 지역 주민을 조사해 주소별 사망자 수를 검은 막대로 표시한 지도 제작.
- 📘 지도가 보여준 패턴: 중앙 우물(water pump)에서 멀수록 사망자가 적음 → 펌프가 오염원.
- 📘 결과: 정부가 물을 통한 전염 인정 → 정수 시설 건설, 물 끓여 마시기 지시 → 런던 콜레라 종식.
- 💡 핵심: “지도(위치 데이터) 시각화가 가설을 증명하고 정책을 바꾼 사례”.

7. 사례 ④ 코로나 시뮬레이션 (Washington Post) (슬라이드 20~23)

- 📘 그래픽 기자 Harry Stevens: 가상 바이러스로 사회적 거리두기 효과 시뮬레이션.
- 📘 가정: 건강한 사람도 감염자와 접촉만 해도 감염 / 완전히 회복한 사람은 전파·재감염 없음 / 점 하나 = 한 사람 / 점은 무작위 방향 이동 / 감염자 1명에서 시작 / 인구 200명.
- 📘 빨간 곡선(감염자 수)이 가파르게 상승하다가 회복자가 늘면서 감소.
- 📘 4가지 시나리오: Free-for-all / Attempted quarantine / Moderate distancing / Extensive distancing → Extensive distancing(강력한 거리두기) 이 가장 효과적.
- 📘 Obama 전 대통령이 트위터로 공유, Washington Post 역사상 가장 많이 읽힌 기사.

8. 적절한 시각화가 중요한 이유 (슬라이드 24)

- 📘 시각화는 이미 일상에 통합 (날씨, 교통 정보, 선거 결과 …)

- 📘 데이터가 많고 복잡해질수록 패턴·통찰을 이해하는 것의 중요성 증가
- 📘 전문 수준과 무관하게 누구나 이해 → 데이터 기반 의사결정 촉진
- 📘 하지만 모든 시각화가 직관적이거나 가치 있는 것은 아님 → 시각화 과정이 제대로 설계되지 않았을 때 발생 → 올바른 시각화 기법을 배우는 것이 중요.

9. 참고 영상 (슬라이드 25)

- TED: David McCandless — *The Beauty of Data Visualization*
- TED: Steven Johnson — *How the “ghost map” helped end a killer disease*

DV_04 — Plot 종류와 시각화 라이브러리 (Plots & Libraries 1)

> 표기: 📘 = 슬라이드 / 💡 = 이해용 주석 / ➕ = 슬라이드에 없는 추가 설명 이 강의는 개념 중심 (코드 실습은 DV_05 이후). 시험에서 “어떤 plot을 언제 쓰나 / 어떤 함정이 있나 / 어떤 라이브러리가 무엇에 좋나”로 출제 가능성 높음.

0. Plot 이란? (슬라이드 2~3)

- 📘 데이터를 시각적 형태로 표현 / 정보·추세·통찰을 효과적으로 전달 / 패턴 발견·추세 식별 / 복잡한 정보를 더 쉽게 이해
- 📘 이 강의의 5가지 plot: Line, Bar, Scatter, Box, Histogram

1. 5가지 Plot 한눈에 비교

Plot	한 줄 정의	쓰는 변수(데이터)	대표 용도	주의할 함정
Line	데이터 점들을 직선으로 연결	x: 연속형(주로 시간), y: 연속값	시간에 따른 추세	y축 시작값(0) 에 따라 오해 유발
Bar	직사각형 막대의 높이/길이로 크기 표현	x: 범주형, y: 수치 (가로/세로 모두 가능)	범주 간 비교, 구성 비율, 순위	y축이 0에서 시작 안 하면 왜곡
Scatter	두 변수의 값을 직교좌표 위 점으로	x: 연속형, y: 연속형 (2개의 연속 변수)	상관관계, 군집, 이상치	이상치가 해석에 큰 영향
Box	분포 + 요약 통계 (IQR 상자, 중앙값 선, 수염, 이상치는 점)	y: 연속형, (x: 범주형 그룹)	그룹 간 분포 비교, 퍼짐·왜도, 이상치	이상치를 무시·오처리하면 해석 왜곡
Histogram	구간(bin)별 빈도를 막대로	연속형 변수 1개	분포 모양, 왜도, 변동성	bin 개수 선택에 따라 모양이 달라짐

---

2. Line Plot (슬라이드 4~5)

- 📘 정의: 데이터 점들을 직선으로 이어 표시
- 📘 사용 경우:
  - 시간에 따른 추세 표시
  - 연속적인 독립변수를 가진 데이터셋 비교
  - 인과관계 설명
  - 연속 데이터 시각화
- 📘 함정: 축의 스케일을 신중히 고르지 않으면 오해를 부를 수 있음.
  - 예: 1998년 이민자 감소 그래프 — y축을 0에서 시작하도록 고치기 전(BEFORE) 에는 급락처럼 보이고, 후(AFTER) 에는 완만한 변화.
- 💡 y축을 확대(잘라서)하면 작은 변화가 과장돼 보인다 → Line plot도 y축을 0부터 시작해 보는 습관.

3. Bar Plot (슬라이드 6~8)

- 📘 정의: 직사각형 막대로 데이터 표시, 막대의 높이(길이)가 크기, 세로/가로 모두 가능 (슬라이드 6: 세로 막대 + 가로 막대 예시)
- 📘 사용 경우:
  - 서로 다른 범주/그룹 비교
  - 뚜렷한 범주를 가진 이산(discrete) 데이터 표시
  - 각 범주가 전체에 기여하는 정도 표시
  - 순위/정렬하기 쉬운 데이터 시각화
- 📘 함정: 부적절한 막대 선택이나 축 스케일은 오해를 부르는 plot.
  - 예: 1980~2013 아이슬란드 → 캐나다 이민자 — BEFORE: y축이 6에서 시작(막대 길이 차이가 과장) / AFTER: y축을 0에서 시작하도록 교정.
- 💡 막대 길이 = 값이므로 기준선이 0이어야 비교가 공정. 0이 아닌 곳에서 자르면 길이 비율이 값 비율과 달라짐.

4. Scatter Plot (슬라이드 9~11)

- 📘 정의: 직교(cartesian) 좌표로 두 변수의 값을 표시. 한 변수 값은 가로축 위치, 다른 변수 값은 세로축 위치를 결정.
- 📘 사용 경우:
  - 두 연속형 변수 간 관계 조사
  - 데이터의 패턴·추세 조사
  - 이상치(outlier) 또는 특이한 관측값 탐지
  - 데이터 내 군집(cluster) 식별
- 📘 함정: 이상치가 해석에 큰 영향 → 포함/제외 여부를 고려해야 함. (슬라이드 11: “이상치 있음” vs “이상치 없음” 두 scatter를 비교 — 이상치가 있으면 축 범위가 넓어져 대부분의 점이 한 곳에 뭉쳐 보임.)

5. Box Plot (슬라이드 12~13)

- 📘 정의: 데이터셋의 분포와 주요 통계량을 표시
- 📘 구성: 사분위 범위(IQR) = 상자(box), 중앙값(median) 선, 수염(whiskers) = 범위 표시, 이상치 = 수염 밖의 개별 점
- 슬라이드 12 그림 구성요소: MAXIMUM / UPPER QUARTILE(Q3) / MEDIAN(Q2) / LOWER QUARTILE(Q1) / MINIMUM, 그리고 수염(WHISKERS), 상자(BOX)
- 📘 사용 경우:
  - 범주/그룹별 연속형 변수의 분포 비교
  - 데이터의 퍼짐(spread)과 왜도(skewness) 확인 — 사분위수 시각화
  - 이상치 식별·분석
  - 요약 통계 시각화
  - 데이터셋의 여러 변수 분포 비교
- 📘 박스 플롯은 이상치의 존재와 정도를 알려 줌. 이상치를 무시하거나 잘못 다루면 해석이 왜곡되고 중요한 통찰이 가려짐 (scatter와 동일).
- ➕ IQR = Q3 − Q1. 일반적으로(matplotlib/seaborn 기본) 수염은 Q1 − 1.5×IQR ~ Q3 + 1.5×IQR 범위 안의 가장 먼 데이터까지, 그 밖은 이상치 점으로 표시. 상자가 위/아래로 치우치거나 수염 길이가 비대칭이면 왜도의 단서. (슬라이드에는 1.5×IQR 기준이 없음)

6. Histogram (슬라이드 14~16)

- 📘 정의: 데이터셋 분포의 그래픽 표현. 구간(interval)별 빈도 또는 상대 빈도를 보여줌. 막대 = 각 구간의 데이터 개수. (슬라이드 14 예: Titanic 데이터의 Age Distribution, x=Age, y=Frequency)
- 📘 사용 경우: 데이터 분포 이해 / 데이터의 모양 시각화 / 왜도 평가 / 데이터의 변동성·퍼짐 보여주기
- 📘 함정 — bin(구간) 선택: 난수 데이터로 세 가지 binning 히스토그램 생성 예시
  - Too Few Bins(너무 적음): 모양이 뭉개져 정보 손실
  - Optimal Binning(적절): 분포 모양이 잘 드러남
  - Too Many Bins(너무 많음): 들쭉날쭉해져 노이즈처럼 보임
- 💡 Bar plot과의 차이(➕): Bar = 범주형 변수의 크기 비교(막대 사이 간격 있음) / Histogram = 연속형 변수를 구간으로 나눈 분포(막대가 붙어 있음).

7. 주요 Python 시각화 라이브러리 (슬라이드 17~29)

📘 목록: Pandas, Matplotlib, Seaborn, Folium, Plotly, PyWaffle

Matplotlib (슬라이드 18)

- 📘 범용(general purpose) 플로팅 라이브러리, 다른 라이브러리·프레임워크와 잘 통합
- 📘 특징: 다양한 종류의 plot 생성 / 다양한 요소를 커스터마이징 / 시각화 작업을 강화

Pandas (슬라이드 19~20)

- 📘 데이터 조작(data manipulation) 에 사용. plot 기능은 Matplotlib 기반
- 📘 특징: Pandas 데이터 구조(DataFrame)와 매끄럽게 통합 / 시각화로 탐색적 데이터 분석(EDA)
- 📘 슬라이드 20 코드 예: `df = pd.read_csv('data.csv')` → `df.plot()` → `plt.show()` (matplotlib `pyplot`을 `plt`로 import 후 사용; 칼럼별 선 그래프 4개: Duration, Pulse, Maxpulse, Calories)

Seaborn (슬라이드 21~23)

- 📘 특화된 통계 시각화에 좋음 / 세련된(stylish) plot 제공
- 📘 특징: 다양한 색 팔레트와 스타일 / 여러 plot을 그리드 레이아웃으로 결합하는 함수 / Pandas와 통합
- 📘 슬라이드 22: 팔레트 종류 — pastel, muted, bright, deep, colorblind, dark (가로축 Saturation 채도, 세로축 Luminance 명도). 문서: https://seaborn.pydata.org/tutorial/color_palettes.html

Matplotlib vs Seaborn 비교표 (슬라이드 23, 한글 표)

구분	Matplotlib	Seaborn
표현력	세부 조정 가능 (축, 색상, 선 두께 등 완전 커스터마이징)	통계적 표현(평균, 분포, 회귀선 등) 을 기본 제공
디자인	기본 스타일은 투박함 → 직접 꾸며야 함	기본 디자인이 깔끔 (논문·보고서에 바로 사용 가능)
통계 기능	단순 시각화 도구 (통계 계산은 별도 라이브러리 필요)	boxplot, violinplot, regression plot 등 “통계 시각화” 자동 지원
코드 길이	bar 하나 그려도 코드가 길어짐	같은 시각화를 더 짧은 코드로 구현

구분	Matplotlib	Seaborn
데이터 처리	numpy 배열 위주로 다룸	pandas DataFrame과 친화적 (컬럼 이름만 지정하면 됨)

Folium (슬라이드 24~25)

- 📘 지리공간(geospatial) 데이터 시각화에 탁월 / 인터랙티브하고 커스터마이징 가능한 지도 제작 / 인기 데이터 분석 라이브러리와 통합
- 📘 슬라이드 25: Choropleth Map 이란? ① 지도가 지리적 구역(행정, 정치 등) 을 보여주고 ② 구역을 데이터 값에 따라 색칠(값 범위 0.00~0.09 ⋯ 0.90~0.99 단계별 진하기).

Plotly (슬라이드 26~27)

- 📘 (슬라이드 원문: “Visualizes categorical” — 오타로 보임) 인터랙티브 대시보드 구축 / 웹 브라우저에서 plotting 가능. Dash by Plotly와 함께 표시.
- 📘 예: “Avos Analysis Dashboard” — 월별 막대, 도넛 차트 등 여러 차트를 한 대시보드에.

PyWaffle (슬라이드 28~29)

- 📘 범주형 데이터를 waffle chart로 시각화 / 비율(proportion)을 표현하는 독특한 방식 (정사각형/직사각형 격자). 예: Vehicle Production by Year and Vehicle Type (car / truck / motorcycle)

라이브러리 선택 요약

이럴 때	이 라이브러리
범용 + 세밀한 커스터마이징	Matplotlib
DataFrame 바로 plot, EDA	Pandas (`df.plot()`, Matplotlib 기반)
통계 시각화, 예쁜 기본 디자인	Seaborn
지도 (choropleth 등)	Folium
인터랙티브 / 웹 대시보드	Plotly (+ Dash)
비율을 격자로	PyWaffle

8. Recap (슬라이드 30)

- 📘 Line: 시간에 따른 추세·변화 / Bar: 범주·그룹 비교 / Scatter: 변수 간 관계 / Box: 분포와 이상치 / Histogram: 구간별 데이터 분포
- 📘 Matplotlib: 폭넓은 plot 기능 / Pandas: 데이터 분석과 통합된 plot / Seaborn: 통계 시각화 특화, 매력적인 기본 미학·팔레트 / Folium: 인터랙티브 지도 / PyWaffle: 정사각형·직사각형으로 비율 표현

9. 한눈에: 공통 “함정” 정리 (시험 대비)

Plot	함정	해결
Line	y축 스케일 → 변화 과장/축소	y축 0부터 시작
Bar	y축이 0이 아닌 값에서 시작	y축 0부터 시작
Scatter	이상치	포함/제외 판단
Box	이상치 무시/오처리	이상치 정보를 해석에 활용
Histogram	bin 수 (too few / optimal / too many)	적절한 bin 선택

10. ➕ 다음 실습(DV_05~)을 위한 대응표 (슬라이드 외 추가)

실습 노트에서 plot별 사용 변수를 정리할 때 쓸 틀입니다. 아래는 일반적으로 쓰이는 함수 이름이며, 수업 코드와 다를 수 있으니 실습 파일 받으면 그 코드 기준으로 갱신하겠습니다.

Plot	Matplotlib	Pandas	Seaborn	변수
Line	`plt.plot(x, y)`	`df.plot(kind='line')`	`sns.lineplot(x=, y=, data=)`	x 시간/연속, y 연속
Bar	`plt.bar(x, height)` / `plt.barh`	`df.plot(kind='bar')`	`sns.barplot(x=, y=, data=)`	x 범주, y 수치
Scatter	`plt.scatter(x, y)`	`df.plot(kind='scatter', x=, y=)`	`sns.scatterplot(x=, y=, data=)`	x, y 모두 연속
Box	`plt.boxplot(data)`	`df.plot(kind='box')`	`sns.boxplot(x=, y=, data=)`	y 연속, x 범주(선택)
Histogram	`plt.hist(x, bins=)`	`df.plot(kind='hist')`	`sns.histplot(x=, bins=)`	연속 1개

DV_05 — Matplotlib 소개 + Line Plot 실습

> 표기: 📘 = 슬라이드 / 💻 = Colab 실습 노트북 / 💡 = 이해용 주석 / ➕ = 추가 설명 자료:
> `DV_05_matplotlib.pdf`(이론+Lab 안내) + `DV_05_code_matplotlib_line_plots.ipynb`(실습)

PART A. 이론 (슬라이드)

A-1. Matplotlib 소개 (슬라이드 2)

- 📘 John Hunter(미국 신경생물학자)가 만든 Python 데이터 시각화 라이브러리
- 📘 원래 EEG·ECoG(뇌파) 신호 시각화를 위해 개발
- 📘 Hunter의 팀은 단일 라이선스의 독점 소프트웨어를 썼고, 이를 넓게 쓰기 위해 Matlab 기반 대안을 만들게 됨
- 📘 Matlab처럼 쉽게 그래픽을 생성하는 도구로 발전
- 💡 그래서 `pyplot` 함수들이 Matlab 명령어 스타일을 닮았음 (“command style functions”).

A-2. Matplotlib 아키텍처 — 3개 레이어 (슬라이드 3~8)

레이어	역할	비유(슬라이드 4)
Scripting Layer (pyplot)	작업과 plot 생성을 단순화하는 가벼운 스크립팅 인터페이스	가볍다
Artist Layer (Artist)	화면의 모든 요소를 객체로 다룸	다른 개발자와 공유하는 무거운 프로그래밍 패러다임
Backend Layer (FigureCanvas, Renderer, Event)	실제로 그리고 입력을 처리	가장 밑바닥

- 💡 위(사용자와 가까움) → 아래(하드웨어와 가까움) 순: Scripting → Artist → Backend. 이 수업은 Scripting Layer(pyplot) 를 사용.

Backend Layer (슬라이드 5) — 3개의 내장 추상 인터페이스 클래스

1. FigureCanvas (`matplotlib.backend_bases.FigureCanvas`): figure가 그려지는 영역
2. Renderer (`matplotlib.backend_bases.Renderer`): FigureCanvas 위에 그리는 방법을 아는 객체
3. Event (`matplotlib.backend_bases.Event`): 키보드, 마우스 클릭 등 사용자 입력 처리

Artist Layer (슬라이드 6)

- 📘 핵심 객체는 Artist 하나. Artist는 Renderer를 사용해 canvas에 그리는 법을 안다.
- 📘 제목, 선, 눈금 라벨, 이미지 등이 각각 개별 Artist 인스턴스
- 📘 Artist 두 종류:
  - Primitive(기본 요소): 선 `Line2D`, 텍스트 `Text`, 사각형 `Rectangle`
  - Composite(복합 요소): 다른 Artist를 포함·관리 — `Axis`, `Tick`, `Axes`, `Figure`
- 📘 복합 Artist는 다른 복합 Artist와 기본 Artist 모두 포함할 수 있음.

Scripting Layer (슬라이드 7~8)

- 📘 주로 pyplot. Artist layer보다 가벼운 스크립팅 인터페이스
- 📘 예: 난수 10,000개 히스토그램(슬라이드 7), 슬라이드 8 제목은 100개로 표기

```python
import matplotlib.pyplot as plt     # pyplot을 plt로 import (빨간 박스)
import numpy as np
x = np.random.randn(10000)          # 표준정규분포 난수 10000개
plt.hist(x, 100)                    # 히스토그램, bin 100개 (파란 박스)
plt.title(r'Normal distribution with $\mu=0, \sigma=1$')
plt.savefig('matplotlib_histogram.png')   # 이미지로 저장
plt.show()
```

- 💡 이 코드가 pyplot 사용의 기본 틀: import → 데이터 → plot 함수 → 꾸미기(title) → 저장/표시(show).

A-3. Anatomy of a figure (슬라이드 9)

figure를 구성하는 요소 이름 (https://matplotlib.org/stable/gallery/showcase/anatomy.html) — 그림에 표기된 요소와 대응 코드:

요소	의미	대응 코드(그림 표기)
Figure	전체 그림	`plt.figure`
Axes	그래프가 그려지는 영역	`fig.subplots`
Title	제목	`ax.set_title`
Legend	범례 (예: Blue signal / Orange signal)	`ax.legend`
Grid	격자	`ax.grid`
Line	선	`ax.plot`
Markers	점 표시	`ax.scatter`
Spine	축 테두리선	`ax.spines`
X Axis / Y Axis	축	`ax.xaxis` / `ax.yaxis`
x label / y label (Axis label)	축 이름	`ax.set_xlabel` / `ax.set_ylabel`
Major tick / Minor tick	주 눈금 / 보조 눈금	`ax.xaxis.set_major_locator` / `set_minor_locator`
Major tick label / Minor tick label	눈금 글자	`ax.xaxis.set_major_formatter` / `set_minor_formatter`

A-4. Recap (슬라이드 10)

- Matplotlib은 Python에서 가장 널리 쓰이는 시각화 라이브러리 중 하나
- 초기에는 EEG/ECoG 시각화 도구로 개발
- 3개 레이어: Backend / Artist / Scripting
- plot의 anatomy = 데이터의 시각적 표현을 이루는 여러 구성 요소
- 더 읽기: https://aosabook.org/en/v2/matplotlib.html

A-5. Lab 세션 안내 (슬라이드 11~16)

가장 기본적인 plot — 점 하나 (슬라이드 11~12)

```
import matplotlib.pyplot as plt
plt.plot(5, 5, 'o')     # 점 (5,5), 'o'는 원 모양 marker
plt.show()
```

- 📘 **`%matplotlib inline`** 을 첫 줄에 쓰면 Jupyter/Colab 안에서 그래프가 inline으로 표시(슬라이드 12 빨간 박스)
- 📘 라벨·제목은 **`plt.show()`** 앞에 추가해야 함:

```
plt.plot(5, 5, 'o')
plt.ylabel('Y')
plt.xlabel('X')
plt.title('Plotting Example')
plt.show()          # ← 꾸미는 줄은 반드시 show() 전에!
```

데이터셋 (슬라이드 14~15)

- 📘 캐나다 이민(Immigration to Canada) 데이터셋
- 📘 UN 인구국(Population Division) 이 45개국 자료를 모음
- 📘 국가별 연간 국제 이주 흐름 + 기타 메타데이터
- 📘 이 수업에서는 캐나다로의 이민 UN 데이터를 주로 사용 (슬라이드 15는 원본 엑셀 화면: Type, Coverage, OdName, AREA, AreaName, REG, RegName, DEV, DevName, 1980~ 연도 열)

Line plot 이론 (슬라이드 16)

- 📘 line chart/plot = 정보를 'markers'라 불리는 데이터 점들을 직선 구간으로 연결해 보여주는 plot. 여러 분야에서 쓰는 기본 차트
- 📘 연속적인 데이터셋일 때 사용. 일정 기간에 걸친 추세(trend) 기반 시각화에 최적
- 📘 언제 쓰나(When to use): 시간에 따른 추세·변화 시각화 / 관계 보여주기 / 여러 데이터 시리즈 비교 / 급격한 변화·이상 현상 강조

PART B. Colab 실습 — Line Plot (노트북 `DV_05_code_matplotlib_line_plots.ipynb`)

B-0. 전체 순서 요약 (이것만 보면 흐름이 잡힘)

```
① import numpy, pandas
② pd.read_csv() 로 데이터 읽기
③ df.head() 로 확인
④ df.set_index('Country', inplace=True)  → 국가명을 index로
⑤ (선택) df.index.name = None
⑥ %matplotlib inline, import matplotlib as mpl / matplotlib.pyplot as plt
⑦ (선택) mpl.style.use(['ggplot'])
⑧ years = list(map(str, range(1980, 2013+1)))   # 연도 문자열 리스트
⑨ 데이터 뽑기: df_can.loc[행(국가), 열(연도)]
⑩ .plot(kind='line')  (+ transpose 필요 여부 확인)
⑪ plt.title / ylabel / xlabel / text
⑫ plt.show()
```

B-1. 준비 (셀 3~13)

```
        import numpy as np   # 과학 계산
        import pandas as pd  # 주 데이터 구조 라이브러리

        df_can = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/.../Canada.csv')
        # df_can = pd.read_csv('DV_05_data_canada_immigration.csv')   # (대안) 로컬 CSV 파일
        print('Data read into a pandas dataframe!')

        df_can.head()                          # 앞 5행 확인 (df_can.head(10) 처럼 행 수 지정 가능)

        df_can.set_index('Country', inplace=True)   # Country 열을 index로 (inplace=True: 원본을 직접
변경, 반환값 None)
        df_can.index.name = None                    # (선택) index 이름 제거
```

- 💡 왜 Country를 index로? → `df.loc['Haiti', ...]` 처럼 국가 이름으로 바로 행을 찾을 수 있고, plot 시 국가명이 범례에 자동으로 들어감.
- 💡 데이터 구조: 행 = 국가(Country), 열 = Continent, Region, DevName, 1980~2013(연도), Total.
- 📘 노트북 주석: “Recall that we created a Total column” — 국가별 누적 이민자 수(Total) 열이 CSV에 이미 포함되어 있음 (DV_06에서 set_index 후 열 38개 = 메타 3 + 연도 34 + Total 1로 확인).

B-2. Matplotlib 불러오기 (셀 14~21)

```
        %matplotlib inline                  # inline backend 사용
        import matplotlib as mpl
        import matplotlib.pyplot as plt

        print('Matplotlib version: ', mpl.__version__)   # 버전 확인 (>= 2.0.0), 출력 예: 3.10.0
        print(plt.style.available)                       # 사용 가능한 스타일 목록
        mpl.style.use(['ggplot'])                        # (선택) ggplot 스타일 적용
```

- 📘 `matplotlib.pyplot` = Matplotlib의 scripting layer. MATLAB처럼 동작하는 command style 함수 모음.
- 📘 각 pyplot 함수는 figure에 변화를 줌: figure 생성 → figure 안에 plotting 영역 생성 → 선 그리기 → 라벨 등으로 꾸미기.
- 💡 이번 실습은 scripting layer만 사용, Artist layer는 이후 실습에서.
- 사용 가능 스타일 예: Solarize_Light2, bmh, classic, dark_background, fast, fivethirtyeight, ggplot …

B-3. pandas로 plot 그리기 (셀 22)

- 📘 pandas는 Matplotlib이 내장되어 있어, Series/DataFrame 뒤에 **`.plot()`** 만 붙이면 됨.

B-4. 사례: 아이티 지진 Line plot (셀 24~34)

> 2010년 아이티 규모 7.0 지진 → 캐나다가 난민 수용 확대 → 이민자 급증을 line plot으로 확인

1) 연도 리스트 만들기 / 데이터 Series 추출

```
        years = list(map(str, range(1980, 2013+1)))   # ['1980','1981',...,'2013']  ← 열 이름이 문자열이라 문자열로!
        haiti = df_can.loc['Haiti', years]            # 행: 'Haiti', 열: 1980~2013 (Total 열 제외)
```

- 💡 `range(1980, 2013+1)`: 끝 값은 포함 안 되므로 +1 해야 2013까지.
- 💡 `df.loc[행, 열]`: index 이름으로 선택. 결과는 Series(연도 = index, 값 = 이민자 수).

2) 기본 plot

```
        haiti.plot()              # 반환: <Axes: >
```

- 📘 pandas는 x축에 index 값(연도), y축에 열 값(이민자 수) 을 자동으로 채움.

3) 제목·축 이름 붙이기

```
        haiti.plot(kind='line')
        plt.title('Immigration from Haiti')
        plt.ylabel('Number of immigrants')
        plt.xlabel('Years')
        plt.show()          # 변경 사항을 보려면 반드시 필요
```

4) 텍스트 주석 **`plt.text()`**

```
        haiti.index = haiti.index.map(int)     # index(연도)를 문자열 → 정수로 변환
        haiti.plot(kind='line')
        plt.title('Immigration from Haiti')
        plt.ylabel('Number of Immigrants')
        plt.xlabel('Years')
```

```
plt.text(2000, 6000, '2010 Earthquake')   # 문법: plt.text(x, y, label)
plt.show()
```

- 📘 x좌표 지정법 주의:
  - 연도가 정수(int) → x = 실제 연도 값 → `plt.text(2000, 6000, ...)`
  - 연도가 문자열(string) → x = index 위치(순번) → 1980 기준 20번째가 2000 → `plt.text(20, 6000, ...)`
- 💡 y축은 정수이므로 그냥 값(6000)을 지정.
- 💡 2010년부터 급증 → 지진과 난민 수용 확대의 영향. (고급 주석 방법은 이후 모듈)

B-5. 여러 나라 비교: India vs China (셀 35~48)

Step 1. 두 나라 데이터 추출

```
df_CI = df_can.loc[['India', 'China'], years]   # 국가 여러 개 = 리스트로 전달!
df_CI
```

Step 2. 그냥 plot 하면 이상하게 나옴

```
df_CI.plot(kind='line')       # ❌ 의도와 다르게 나옴 ("That doesn't look right...")
```

- 💡 이유: pandas는 index → x축, 각 열(column) → 하나의 선으로 그림. 그런데 df_CI는 index = 국가, 열 = 연도라서 축이 뒤집힘.

Step 3. 전치(transpose)

```
df_CI = df_CI.transpose()     # 행/열 교환 → index=연도, 열=India, China
df_CI.head()
```

Step 4. 다시 plot

```
df_CI.index = df_CI.index.map(int)     # 연도 index를 정수로 (plot용)
df_CI.plot(kind='line')
plt.title('Immigrants from China and India')
plt.ylabel('Number of Immigrants')
plt.xlabel('Years')
plt.show()
```

- 📘 `kind` 파라미터로 plot 종류를 명시적으로 지정.
- 📘 결과 해석: 중국과 인도는 연도별 이민 추세가 매우 비슷.

❓ 아이티는 왜 transpose 안 했나? (셀 48)

- `haiti`는 Series라 이미 연도가 index임. (DataFrame이 아니라서 불필요)

```
print(type(haiti))     # <class 'pandas.core.series.Series'>
```

- 💡 핵심 규칙: *Series: index가 x축 / DataFrame: index = x축, 각 열 = 한 줄 → x축에 올 것(연도)이 index가 되도록 맞춰라.*

B-6. 이민 상위 5개국 추세 (셀 49~52)

- 📘 line plot은 하나의 독립변수에 대해 여러 종속변수를 보여주기 좋음. 단, 한 그래프에 5~10개 선 이하 권장 (더 많으면 해석 어려움).

```
    # Step 1: 데이터 — Total 열로 정렬해서 상위 5개국
    df_can.sort_values(by='Total', ascending=False, axis=0, inplace=True)   # 내림차순, 원본 변경
    df_top5 = df_can.head(5)                        # 상위 5개
    df_top5 = df_top5[years].transpose()            # 연도 열만 선택 + 전치
    print(df_top5)

    # Step 2: plot — figsize로 크기 키우기
    df_top5.index = df_top5.index.map(int)
    df_top5.plot(kind='line', figsize=(14, 8))    # (가로, 세로) 튜플
    plt.title('Immigration Trend of Top 5 Countries')
    plt.ylabel('Number of Immigrants')
    plt.xlabel('Years')
    plt.show()
```

- 💡 `sort_values(by=정렬할 열, ascending=False(내림차순), axis=0(행 기준), inplace=True)`
- 💡 상위 5개국 중 India, China, United Kingdom of Great Britain and Northern Ireland 등이 포함 (출력 열 확인).

PART C. Line Plot 정리 — “plot별 사용 변수” 표

실습	데이터	x축	y축	함수/옵션
Haiti	Series `df_can.loc['Haiti', years]`	연도 (index)	이민자 수	`haiti.plot(kind='line')`
India vs China	DataFrame `df_can.loc[['India','China'], years]` → transpose	연도 (index)	국가별 이민자 수(열 = 선)	`df_CI.plot(kind='line')`
Top 5	`sort_values('Total')` → `head(5)` → `[years].transpose()`	연도 (index)	국가별 이민자 수	`plot(kind='line', figsize=(14,8))`

꾸미기 함수 모음

함수	역할
`plt.title('...')`	제목
`plt.xlabel('...')` / `plt.ylabel('...')`	축 이름
`plt.text(x, y, '...')`	좌표 (x, y)에 텍스트 주석
`plt.show()`	그림 표시 (꾸미는 코드는 이 앞에)
`plt.savefig('name.png')`	파일 저장 (슬라이드 예시)
`mpl.style.use(['ggplot'])`	스타일 적용
`plot(kind='line', figsize=(w, h))`	plot 종류, 그림 크기

자주 틀리는 포인트 (시험 대비)

1. 연도 열 이름은 문자열(`'1980'`) → `years = list(map(str, range(1980, 2014)))`
2. 여러 국가 선택은 리스트: `.loc[['India','China'], years]`
3. 국가별 선을 그리려면 transpose (index=x축, 열=선)
4. `plt.text()` 의 x값: 연도가 int면 연도 값, str이면 순번
5. 라벨/제목은 **`plt.show()`** 전에
6. 선은 5~10개 이하
7. `inplace=True`는 원본 변경(반환 None)

DV_06 — Area Plot · Histogram · Bar Chart (Matplotlib + pandas)

> 표기: 📘 = 슬라이드 / 💻 = Colab 실습 노트북 / 💡 = 이해용 주석 / ➕ = 추가 설명 자료:
> `DV_06_Area_Hist_Bar_plot.pdf` + `DV_06_Area_Hist_Bar.ipynb` (캐나다 이민 데이터셋, DV_05와 동일)

PART A. 이론 (슬라이드)

A-1. Area Plot (슬라이드 2, 8~9)

- 📘 area chart / area graph 라고도 함
- 📘 여러 변수의 크기(magnitude)와 비율(proportion) 을 표시
- 📘 시간 또는 다른 순서가 있는 차원을 나타냄
- 📘 line plot을 기반으로 함 (선 아래를 색으로 채운 것)
- 📘 용도: 주식 시장 성과 추적 / 인구통계(demographics) 시각화 / 자원 분포 표시
- 슬라이드 8 예시 코드와 그림: `df_top5.plot(kind='area')` — 상위 5개국(India, China, United Kingdom, Philippines, Pakistan)의 누적 영역 그래프

A-2. Histogram (슬라이드 10)

- 📘 데이터셋의 빈도 분포(frequency distribution) 를 표현
- 📘 숫자 데이터를 bin(구간)으로 분할 → 각 데이터 포인트를 bin에 할당 → 각 bin의 개수를 센다
- 슬라이드 12: `np.histogram`의 bin 표 — Bin 1(0~3412.9)=178, Bin 2(3412.9~6825.8)=11, Bin 3(~10238.7)=1, Bin 4(~13651.6)=2, Bin 5~8=0, Bin 9(27303.2~30716.1)=1, Bin 10(30716.1~34129)=2
- 슬라이드 14 코드: `import numpy as np` + `count, bin_edges = np.histogram(df['2013'])` + `plot(kind='hist', xticks=bin_edges)` (빨간 박스 3곳이 핵심)

A-3. Bar Chart (슬라이드 15~18)

- 📘 bar graph 라고도 함 / 변수의 값을 비교
- 슬라이드 16: Iceland 이민자 세로 막대 `df_iceland.plot(kind='bar')`
- 슬라이드 17: 가로 막대 `df_iceland.plot(kind='barh', color='blue')`
- 슬라이드 18: 특정 막대만 색 강조

```
        c = ['blue','red','blue','blue','blue','blue','blue','blue','blue','blue','red']   # 막대 개수만큼의 색 리스트
        df_Iceland.plot(kind='bar', color=c, edgecolor='black')   # 1980~1990 → 1981, 1990만 빨강
```

- 📘 요약: 막대 길이는 대표하는 값에 비례. 강조하고 싶은 막대는 파라미터로 색을 바꿔 강조.

A-4. Histogram vs Bar (슬라이드 19~20) ⭐ 시험 단골 비교

	Histogram	Bar Chart
데이터 종류	연속형(continuous)	범주형(categorical)
막대 의미	구간(bin) 내 빈도	범주 하나의 값 또는 빈도
막대 사이 간격	없음 (연속이라서)	있음 (범주가 구분되므로)
순서	순서 유지 필수	순서 바꿀 수 있음

	Histogram	Bar Chart
목적	분포(distribution) 보기	범주 비교(compare categories)
예시	시험 점수 분포, 키 분포 (“Scores 0–10, 10–20, …”)	전공별 학생 수, 제품 유형별 매출 (“A, B, C, D, F 학점”)

A-5. Recap (슬라이드 21)

- Area plot: 시간에 따른 누적 합계를 숫자 또는 퍼센트로 표현. Matplotlib import 후 DataFrame에 `plot()` 호출, **`kind='area'`**. 한 차트에서 여러 변수의 관계와 비율을 직관적으로 보여줌
- Histogram: 수치 데이터의 빈도 분포. `mpl`, `plt` import, DataFrame에 **`kind='hist'`**. Numpy로 bin을 만들어 사용 가능
- Bar chart: 막대 길이가 항목 값에 비례. 강조하려는 막대의 색을 파라미터로 변경

PART B. Colab 실습 (노트북 `DV_06_Area_Hist_Bar.ipynb`)

B-0. 전체 순서 요약

```
[공통 준비]
① import numpy, pandas, %matplotlib inline, matplotlib(mpl), pyplot(plt), mpl.style.use('ggplot')
② pd.read_csv() → df_can
③ df_can.head(), df_can.shape
④ df_can.set_index('Country', inplace=True)
⑤ years = list(map(str, range(1980, 2014)))

[Area]  sort_values('Total') → head() → [years].transpose() → index를 int로 → plot(kind='area',
stacked=, alpha=, figsize=)
[Hist]  np.histogram으로 bin 확인 → plot(kind='hist', bins=, alpha=, xticks=bin_edges, color=,
stacked=, xlim=)
[Bar]   .loc[국가, years] → plot(kind='bar'/'barh', ...) → plt.annotate()로 화살표/글자
[비교]  점수 데이터 생성 → 학점 범주화 → 히스토그램 vs 막대 나란히
```

B-1. 공통 준비 (셀 1~13)

```
import numpy as np
import pandas as pd
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
mpl.style.use('ggplot')                                  # (선택) ggplot 스타일
print('Matplotlib version: ', mpl.__version__)       # 3.10.0

df_can = pd.read_csv('https://...Canada.csv')
df_can.head()
print(df_can.shape)                     # (195, 39)  → 195개국, 39개 열
df_can.set_index('Country', inplace=True)
print('data dimensions:', df_can.shape)   # (195, 38)  → Country가 index로 빠져 열이 38개
years = list(map(str, range(1980, 2014)))   # '1980'..'2013'  (끝 2014 = 2013까지)
```

- 💡 열 38개 = Continent, Region, DevName + 1980~2013 (34개) + Total → **`Total`** 열은 CSV에 이미 들어 있음 (이전 DV_05 노트의 “Total 생성 셀 없음”은 이것으로 해결: 데이터에 포함).
- 💡 슬라이드 3~4: 원본 표는 Type, Coverage, OdName, AREA, AreaName, REG, RegName, DEV, DevName, 연도들; 정리 후 df_canada는 Country가 index(빨간 박스), Total 열(초록 박스) 가 마지막 열.

B-2. Area Plot

데이터 준비 (셀 16)

```
df_can.sort_values(['Total'], ascending=False, axis=0, inplace=True)   # Total 내림차순
df_top5 = df_can.head()                        # 상위 5개 (head 기본값 5)
```

```
df_top5 = df_top5[years].transpose()          # 연도 열만 + 전치 (index=연도, 열=국가)
```

핵심 개념

- 📘 Area plot은 기본이 stacked(누적)
- 📘 stacked area는 각 열이 모두 양수 또는 모두 음수여야 함 (NaN은 0으로 처리)
- 📘 unstacked로 그리려면 **`stacked=False`**
- 📘 unstacked의 기본 투명도(alpha) = 0.5, `alpha`로 변경 (0~1)

코드 3종 (셀 18~22)

```
df_top5.index = df_top5.index.map(int)      # 연도 index를 정수로

# ① unstacked (겹쳐서)
df_top5.plot(kind='area', stacked=False, figsize=(20, 10))
# ② stacked (쌓아서)
df_top5.plot(kind='area', stacked=True,  figsize=(20, 10))
# ③ stacked + 투명도
df_top5.plot(kind='area', alpha=0.25, stacked=True, figsize=(20, 10))

plt.title('Immigration Trend of Top 5 Countries')
plt.ylabel('Number of Immigrants')
plt.xlabel('Years')
plt.show()
```

- 💡 stacked=True: 각 나라 영역이 위로 쌓임 → 전체 합과 각 나라의 몫(기여도) 을 봄 / stacked=False: 각 나라 영역이 같은 바닥(0)에서 시작해 겹침 → 나라 간 직접 비교, 겹침이 보이도록 투명도 사용.

★ B-3. 두 가지 plotting 스타일 (셀 23~27) — 개념 이해 필수

	Option 1: Scripting layer (procedural)	Option 2: Artist layer (object-oriented, **preferred**)
사용	`plt` (= `matplotlib.pyplot`) 함수 호출	plot이 반환하는 Axes 객체(**`ax`**) 에 메서드 호출
제목	`plt.title(...)`	`ax.set_title(...)`
x 라벨	`plt.xlabel(...)`	`ax.set_xlabel(...)`
y 라벨	`plt.ylabel(...)`	`ax.set_ylabel(...)`
의미	"현재 활성화된 plot의 제목 설정" — plot 하나일 때 편리	"**`ax`**가 나타내는 plot의 제목 설정" — 어느 plot인지 명확, 여러 plot일 때 유리, 고급 plot에 유연

```
# Option 1
df_top5.plot(kind='area', alpha=0.35, figsize=(20, 10))
plt.title('Immigration trend of top 5 countries')
plt.ylabel('Number of immigrants')
plt.xlabel('Years')

# Option 2
ax = df_top5.plot(kind='area', alpha=0.35, figsize=(20, 10))   # plot 영역(Axes)을 ax에 저장
```

```
        ax.set_title('Immigration Trend of Top 5 Countries')
        ax.set_ylabel('Number of Immigrants')
        ax.set_xlabel('Years')
```

- 💡 규칙: plt.XXX → ax.set_XXX ("set_"을 붙임).
- 📘 이 수업은 scripting layer 위주, 고급 시각화에서만 artist layer 사용.

연습 문제 ① (셀 28~30) 스크립팅 레이어: **가장 적게 기여한 5개국** stacked area, alpha=0.45

```
        df_least5 = df_can.tail(5)                          # Total 내림차순 정렬 상태이므로 tail = 최하위 5개국
        df_least5 = df_least5[years].transpose()
        df_least5.index = df_least5.index.map(int)
        df_least5.plot(kind='area', alpha=0.45, figsize=(20, 10))   # stacked 기본값 True
        plt.title('Immigration Trend of 5 Countries with Least Contribution to Immigration')
        plt.ylabel('Number of Immigrants')
        plt.xlabel('Years')
        plt.show()
```

연습 문제 ② (셀 31~33) 아티스트 레이어: 최하위 5개국 **unstacked**, alpha=0.55

```
        ax = df_least5.plot(kind='area', alpha=0.55, stacked=False, figsize=(20, 10))
        ax.set_title('Immigration Trend of 5 Countries with Least Contribution to Immigration')
        ax.set_ylabel('Number of Immigrants')
        ax.set_xlabel('Years')
```

- 💡 상위 = **`head()`**, 하위 = **`tail()`** (단, `Total` 내림차순 정렬이 되어 있을 때).

B-4. Histogram

개념 (셀 34)

- 📘 x축을 bin으로 분할 → 각 데이터 포인트를 bin에 할당 → bin별 개수를 셈. y축 = 빈도(각 bin의 데이터 수). bin 크기는 바꿀 수 있고, 분포가 잘 보이도록 조정 필요.

질문 1: 2013년 신규 이민자 수(195개국)의 빈도 분포 (셀 35~44)

```
        df_can['2013'].head()                               # 확인 (Series: index=국가, 값=2013 이민자 수)

        count, bin_edges = np.histogram(df_can['2013'], bins=10)   # np.histogram은 2개 값 반환!
        print(count)       # [178  11   1   2   0   0   0   0   1   2]   ← 빈도
        print(bin_edges)   # [0. 3412.9 6825.8 ... 34129.]              ← bin 경계 (기본 10 bins → 경계 11개)
```

- 📘 178개국이 0~3412.9명, 11개국이 3412.9~6825.8명, 1개국이 6825.8~10238.7명 … 기여.

```
        df_can['2013'].plot(kind='hist', figsize=(8, 5))
        plt.title('Histogram of Immigration from 195 Countries in 2013')
```

```
        plt.ylabel('Number of Countries')     # y = 나라 수 (빈도)
        plt.xlabel('Number of Immigrants')    # x = 이민자 수 구간
        plt.show()
```

- 문제점 📘: x축 눈금이 bin 크기와 안 맞음 → **`xticks=bin_edges`** 로 해결

```
        count, bin_edges = np.histogram(df_can['2013'], bins=10)
        df_can['2013'].plot(kind='hist', figsize=(8, 5), xticks=bin_edges)
```

- 📘 Side note: `data.plot(kind='hist')` ≡ `data.plot.hist()` (kind 인자 방식과 메서드 방식은 동일, 모든 plot 종류에서 성립)

질문 2: 덴마크·노르웨이·스웨덴 1980~2013 분포 (셀 45~55)

```
        df_can.loc[['Denmark', 'Norway', 'Sweden'], years].plot.hist()     # ❌ 이상한 그래프
```

- 📘 이유: 3개국의 이민자 분포가 아니라 연도(years)별 분포를 그림. 데이터 구조(행/열) 문제.
- ✅ 해결: transpose 후 plot

```
        df_t = df_can.loc[['Denmark', 'Norway', 'Sweden'], years].transpose()   # 열 = 국가, 행 = 연
도
        df_t.plot(kind='hist', figsize=(10, 6))
        plt.title('Histogram of Immigration from Denmark, Norway, and Sweden from 1980 - 2013')
        plt.ylabel('Number of Years')            # y = 해당 구간에 속한 "연도 수"
        plt.xlabel('Number of Immigrants')
        plt.show()
```

- 💡 y축 의미가 다름: 질문 1의 y = “나라 수”(행=국가 195개), 질문 2의 y = “연도 수”(행=연도 34개). 히스토그램의 y = 행(관측값)의 개수 → 행에 무엇이 오는지가 중요.

꾸미기: bins, alpha, xticks, color (셀 51~52) — 겹친(unstacked) 히스토그램

```
        count, bin_edges = np.histogram(df_t, 15)        # 15 bins로 경계 계산 (xticks용)

        df_t.plot(kind='hist',
                  figsize=(10, 6),
                  bins=15,                                # bin 개수 15
                  alpha=0.6,                              # 투명도 60%
                  xticks=bin_edges,                       # x축 눈금 = bin 경계
                  color=['coral', 'darkslateblue', 'mediumseagreen'])   # 열(국가)별 색
```

- 📘 노트북 설명: bin 크기 15 / 투명도 60% / (x-label) / 색 변경은 **`color`** 파라미터
- 📘 Tip: Matplotlib의 전체 색 목록 → `matplotlib.colors.cnames.items()` 순회 출력

겹치지 않게: stacked + xlim (셀 54~55)

```
        count, bin_edges = np.histogram(df_t, 15)
        xmin = bin_edges[0] - 10      # 첫 bin 값(31.0) 앞에 여백 10
        xmax = bin_edges[-1] + 10     # 마지막 bin 값(308.0) 뒤에 여백 10

        df_t.plot(kind='hist', figsize=(10, 6), bins=15, xticks=bin_edges,
```

```
                color=['coral', 'darkslateblue', 'mediumseagreen'],
                stacked=True,                 # 쌓기 (겹치지 않음)
                xlim=(xmin, xmax))            # x축 범위 (min, max) 튜플 → 양 끝 여백 조절
```

- 📘 `stacked` = 서로 겹치지 않게 쌓기 / `xlim=(min, max)` = 양쪽 가장자리의 불필요한 빈 공간 제거
- 💡 bin_edges[0] = 첫 경계, bin_edges[-1] = 마지막 경계.

연습 문제 (셀 56~58): 그리스·알바니아·불가리아, **겹침(overlapping) 15 bins, alpha=0.35**

```
df_cof = df_can.loc[['Greece', 'Albania', 'Bulgaria'], years].transpose()
count, bin_edges = np.histogram(df_cof, 15)
df_cof.plot(kind='hist', figsize=(10, 6), bins=15, alpha=0.35,
            xticks=bin_edges, color=['coral', 'darkslateblue', 'mediumseagreen'])
plt.title('Histogram of Immigration from Greece, Albania, and Bulgaria from 1980 - 2013')
plt.ylabel('Number of Years')
plt.xlabel('Number of Immigrants')
plt.show()
```

B-5. Bar Chart

개념 (셀 59~60)

- 📘 bar plot: 막대의 길이 = 변수의 크기. 보통 수치형·범주형 변수를 구간으로 묶어 표현.
- 📘 `plot()`의 **`kind`**: **`'bar'`** = 세로(vertical) / **`'barh'`** = 가로(horizontal)
- 📘 세로 막대: x축 = 라벨, y축 막대 길이 = 값. 시계열 분석에 유용. 단점: 각 막대 아래 글자 라벨 공간이 부족.
- 📘 가로 막대: y축 = 라벨, x축 길이 = 값. 범주 이름 쓸 공간이 넉넉.

사례: 아이슬란드 금융위기 (2008~2011) — 아이슬란드 → 캐나다 이민

```
df_iceland = df_can.loc['Iceland', years]       # Series
df_iceland.plot(kind='bar', figsize=(10, 6))
plt.xlabel('Year'); plt.ylabel('Number of immigrants')
plt.title('Icelandic immigrants to Canada from 1980 to 2013')
plt.show()
```

- 📘 해석: 2008년 이후 이민자 수가 급증 → 금융위기의 영향.

`plt.annotate()` — 화살표/글자 주석 (셀 63~66)

파라미터	의미
`s`	주석 텍스트 (문자열). 비우면 글자 없음(화살표만)
`xy`	주석할 (x, y) 지점 (= 화살표 머리)
`xytext`	텍스트를 놓을 (x, y) (= 화살표 시작점)
`xycoords`	xy가 쓰는 좌표계. `'data'`(기본) = 주석 대상 객체의 좌표계
`arrowprops`	화살표 속성 딕셔너리: `arrowstyle('->'` 표준 화살표), `connectionstyle('arc3'` = 직선), `color`, `lw`(선 굵기)

파라미터	의미
`rotation`	텍스트 회전각(도, 반시계)
`va`	세로 정렬 `'center' / 'top' / 'bottom' / 'baseline'`
`ha`	가로 정렬 `'center' / 'right' / 'left'`

```
        df_iceland.plot(kind='bar', figsize=(10, 6), rot=90)       # rot=90: x축 눈금 글자를 90도 회전

        plt.xlabel('Year'); plt.ylabel('Number of Immigrants')
        plt.title('Icelandic Immigrants to Canada from 1980 to 2013')

        # (1) 화살표
        plt.annotate('',                          # 글자 없음
                     xy=(32, 70),                 # 화살표 머리: x=32(=2012년), y=70
                     xytext=(28, 20),             # 화살표 꼬리: x=28(=2008년), y=20
                     xycoords='data',
                     arrowprops=dict(arrowstyle='->', connectionstyle='arc3', color='blue', lw=2))

        # (2) 화살표 위 글자
        plt.annotate('2008 - 2011 Financial Crisis',
                     xy=(28, 30),                 # 글자 시작 위치 (2008년, 30)
                     rotation=72.5,               # 화살표 각도에 맞추려 시행착오로 정한 값
                     va='bottom', ha='left')
        plt.show()
```

- 💡 bar plot의 x좌표는 연도 값이 아니라 "막대 순번(0부터)": 1980=0 → 2008=28, 2012=32. (DV_05 `plt.text`에서 문자열 index일 때 순번을 쓰는 것과 같은 원리)
- 💡 `plt.text(x, y, 글자)`는 단순 텍스트, `plt.annotate()`는 화살표까지 가능.

가로 막대 연습 (셀 67~73): 이민 총합 상위 15개국 `barh` + 막대에 숫자 라벨

```
        # Step 1: Total 기준 오름차순 정렬 → 마지막 15개 = 상위 15개
        df_can.sort_values(by='Total', ascending=True, inplace=True)
        df_top15 = df_can['Total'].tail(15)           # Series (국가별 Total)

        # Step 2: 가로 막대 + 값 표시
        df_top15.plot(kind='barh', figsize=(12, 12), color='steelblue')
        plt.xlabel('Number of Immigrants')
        plt.title('Top 15 Conuntries Contributing to the Immigration to Canada between 1980 -
2013')

        for index, value in enumerate(df_top15):            # 각 막대를 순회
            label = format(int(value), ',')                  # 정수에 천 단위 쉼표 (예: 691,904)
            plt.annotate(label, xy=(value - 47000, index - 0.10), color='white')   # 막대 끝 안쪽에
흰 글씨

        plt.show()
```

- 💡 왜 ascending=True + tail(15)? `barh`는 아래에서 위로 그려서 첫 행이 맨 아래에 옴 → 오름차순으로 정렬 후 tail하면 가장 큰 값이 맨 위에 표시됨.
- 💡 `enumerate(df_top15)`: index = 막대 순번(y좌표), value = 값(x좌표). `value - 47000`은 글자를 막대 끝에서 왼쪽(막대 안) 으로 옮겨 넣는 보정.
- 💡 `format(int(value), ',')`: 숫자를 `'691,904'` 형태로.

B-6. Histogram vs Bar 직접 비교 (셀 74~82)

① 예시 데이터 생성 (셀 76)

```
        np.random.seed(42)                                          # 재현 가능하도록 시드 고정
        scores = np.random.normal(loc=70, scale=15, size=200)       # 정규분포: 평균 70, 표준편차 15, 학생
200명
        scores = np.clip(scores, 0, 100)                            # 0~100 범위로 자르기
        df = pd.DataFrame({"Score": scores})
```

② 학점 범주화 (셀 77~79) — bar chart는 **범주화된 데이터**가 필요

- 기준: 90~100 → A / 80~90 → B / 70~80 → C / 60~70 → D / 60 미만 → F

```
        def grade(score):
            if score >= 90: return "A"
            elif score >= 80: return "B"
            elif score >= 70: return "C"
            elif score >= 60: return "D"
            else: return "F"

        df["Grade"] = df["Score"].apply(grade)                      # 각 점수에 함수 적용 → 새 열
        grade_counts = df["Grade"].value_counts().sort_index()      # 학점별 인원 수 (A,B,C,D,F 순 정
렬)
```

③ 나란히 그리기 (셀 80~82) — `plt.subplot`

```
        plt.figure(figsize=(12,5))

        # 오른쪽(subplot 1,2,2): Histogram — 연속형 점수 분포
        plt.subplot(1,2,2)
        plt.hist(df["Score"], bins=10, color="skyblue", edgecolor="black")
        plt.title("Histogram (Continuous: Score Distribution)")
        plt.xlabel("Score"); plt.ylabel("Frequency")

        # 왼쪽(subplot 1,2,1): Bar Chart — 범주형 학점
        plt.subplot(1,2,1)
        plt.bar(grade_counts.index, grade_counts.values, color="lightgreen", edgecolor="black")
        plt.title("Bar Chart (Categorical: Number of Students by Grade)")
        plt.xlabel("Grade"); plt.ylabel("Number of Students")

        plt.tight_layout()      # 겹치지 않게 간격 자동 조정
        plt.show()
```

- 💡 `plt.subplot(행, 열, 위치번호)`: `(1,2,1)` = 1행 2열 중 첫 번째(왼쪽), `(1,2,2)` = 두 번째(오른쪽). 노트북 코드의 주석 순서는 (1) Histogram이 먼저지만 위치는 오른쪽(2번) 이니 헷갈리지 말 것.
- 💡 이 셀은 pandas가 아니라 pyplot 함수(`plt.hist`, `plt.bar`)를 직접 사용: `plt.hist(데이터, bins=, color=, edgecolor=)`, `plt.bar(x(범주), height(값), color=, edgecolor=)`.

PART C. 정리표

C-1. plot별 사용 변수·함수

Plot	사용한 데이터	x축	y축	핵심 코드	주요 파라미터
Area (상위/하위 5개국)	`sort_values('Total')` → `head()`/`tail(5)` → `[years].transpose()`	연도 (index, int)	국가별 이민자 수(열=영역)	`df.plot(kind='area')`	`stacked`(기본 True), `alpha`(0~1), `figsize`
Histogram 1개 열	`df_can['2013']` (Series)	이민자 수 구간 (bin)	나라 수(빈도)	`plot(kind='hist')`	`xticks=bin_edges`, `bins`
Histogram 여러 국가	`df_can.loc[[국가...], years].transpose()`	이민자 수 구간	연도 수(빈도)	`plot(kind='hist')`	`bins`, `alpha`, `xticks`, `color=[...]`, `stacked`, `xlim=(min,max)`
Bar (세로)	`df_can.loc['Iceland', years]` (Series)	연도(범주)	이민자 수	`plot(kind='bar')`	`rot=90`, `color=`, `edgecolor=`
Bar (가로)	`df_can['Total'].tail(15)` (Series)	이민자 총합	국가(범주)	`plot(kind='barh')`	`color`, `figsize`
Hist vs Bar	`df['Score']`(연속), `grade_counts`(범주)	점수 구간 / 학점	빈도 / 학생 수	`plt.hist`, `plt.bar`	`bins`, `edgecolor`, `plt.subplot`

C-2. 파라미터 사전

파라미터	적용 plot	의미
`kind`	전부	plot 종류: `'line' 'area' 'hist' 'bar' 'barh'` (`.plot.hist()` 도 동일)
`stacked`	area, hist	쌓기 여부 (area 기본 True / hist 기본 겹침)
`alpha`	area, hist	투명도 0~1 (area unstacked 기본 0.5)
`figsize=(가로,세로)`	전부	그림 크기
`bins`	hist	bin 개수
`xticks`	hist	x축 눈금 위치 (보통 `bin_edges`)
`xlim=(min,max)`	hist	x축 범위
`color`	hist, bar	색 (한 개 또는 리스트)
`edgecolor`	bar	막대 테두리 색
`rot`	bar	눈금 글자 회전각

C-3. 시험 대비 포인트

1. Area plot은 기본 stacked. 겹치게 하려면 `stacked=False`. 투명도 기본 0.5.
2. 여러 국가를 그리는 plot(area/hist)은 **`transpose()`** 필수 (index=x 또는 관측치, 열=그룹).
3. np.histogram은 (count, bin_edges) 2개 반환. bin_edges는 `xticks=`에 넣어 눈금과 bin을 맞춤.
4. **`plt.xxx()`** ↔ **`ax.set_xxx()`**: scripting vs artist layer. `ax = df.plot(...)`.
5. bar = 범주 / histogram = 연속. 간격 유무, 순서, 목적 차이 설명할 수 있어야 함 (슬라이드 20).
6. **`bar`** = 세로, **`barh`** = 가로. 라벨이 길면 가로.
7. `plt.annotate` 좌표(bar의 x)는 연도가 아니라 막대 순번.
8. 상위 N = `sort_values(내림차순).head(N)`, 하위 N = `.tail(N)`. barh 상위 N을 위에 보이게 하려면 오름차순 + tail(N).
9. 슬라이드 정의: area = line plot 기반, 크기와 비율 표현.

DV_07 — Pie · Box · Scatter · Bubble Plot

> 표기: 📘 = 슬라이드 / 💻 = Colab 실습 노트북 / 💡 = 이해용 주석 / ➕ = 추가 설명 자료:
> `DV_07_pie_box_scatter.pdf` + `DV_07_code_pie_box_scatter_bubble_1.ipynb` (캐나다 이민 데이터셋)

PART A. 이론 (슬라이드)

A-1. Pie Chart (슬라이드 2~5)

- 📘 원형 통계 그래프를 조각(segment)으로 나눈 것. 숫자의 비율(proportion) 을 보여줌.
- 슬라이드 3: `df_continents = df_canada.groupby('Continent', axis=0).sum()` → 대륙별 합계 표 (행 = Africa, Asia, Europe, Latin America and the Caribbean, Northern America, Oceania / 열 = 1980…2013, Total)
- 슬라이드 4: `df_continents['Total'].plot(kind='pie')` + title → 기본 파이 (Asia가 가장 큼, 레이블이 파이 바깥)
- 슬라이드 5: **`explode`** 로 조각을 밖으로 떼어내 강조 + 퍼센트(%) 표시 (예: 51.8%, 22.0%, 11.9%, 9.7%, 3.8%, 0.9%) + 범례를 따로 둠

A-2. Box Plot (슬라이드 6~7)

- 슬라이드 6 그림 — 박스 플롯 구성요소: Maximum, Third Quartile(Q3), Median, First Quartile(Q1), Minimum, 상자 높이 = Inter Quartile Range (IQR), 수염 밖 점 = Outliers
- 슬라이드 7 코드:

```
df_japan = df_canada.loc[['Japan'], years].transpose()
df_japan.plot(kind='box')
plt.title('Box plot of Japanese Immigrants from 1980-2013')
plt.ylabel('Number of Immigrants')
plt.show()
```

A-3. Scatter Plot (슬라이드 8~11)

- 📘 두 변수에 해당하는 값을 표시 / 두 변수 간 상관관계(correlation) 를 파악
- 슬라이드 9 코드: `df_total.plot(kind='scatter', x='year', y='total')` (df_total = year, total 두 열; 1980=99137, 1981=110563, …)
- 슬라이드 10: 결과 — 연도가 갈수록 총 이민자 수 증가 추세
- 슬라이드 11: 점 크기·색을 바꾼 두 scatter (왼쪽 빨간 작은 점, 오른쪽 크기가 다른 파란 점 = bubble 형태)

A-4. Recap (슬라이드 12)

- Pie chart: 원형 통계 그래프를 조각으로 나누어 수치 비율을 나타냄
- Box plot: 데이터 분포를 통계적으로 표현. 5가지 값 = Minimum, 1st quartile, median, 3rd quartile, maximum
- Scatter plot: 보통 두 변수를 서로에 대해 표시
- scatter plot 생성은 Matplotlib을 import해서 큰 데이터 집합을 시각화

PART B. Colab 실습 (노트북 `DV_07_code_pie_box_scatter_bubble_1.ipynb`)

B-0. 전체 순서

```
[준비] import → read_csv → shape → set_index('Country') → years 리스트
[Pie]     groupby('Continent').sum() → ['Total'].plot(kind='pie', autopct, startangle, shadow,
labels, pctdistance, colors, explode) → plt.axis('equal') → plt.legend
[Box]     .loc[['국가'], years].transpose() → describe() → plot(kind='box', vert, color) →
subplot(ax=) → decade 합산 → 이상치(1.5×IQR)
[Scatter] df[years].sum(axis=0) → DataFrame → index int → reset_index → 열 이름 변경 →
plot(kind='scatter', x=, y=) → np.polyfit → 회귀선 → 예측
[Bubble]  df[years].transpose() → Year 열 만들기 → 정규화(min-max) → scatter(s=가중치*2000+10, ax=ax0)
```

B-1. 준비 (셀 2~15)

```
        import numpy as np
        import pandas as pd
        import matplotlib as mpl
        import matplotlib.pyplot as plt          # (이 노트북에서는 %matplotlib inline 주석 처리)
        mpl.style.use('ggplot')
        print('Matplotlib version: ', mpl.__version__)

        df_can = pd.read_csv('DV_04_data_canada_immigration.csv')   # 로컬 CSV 파일 사용 (파일명 주의)
        df_can.head()
        print(df_can.shape)                       # (195, 40)
        df_can.set_index('Country', inplace=True)
        print('data dimensions:', df_can.shape)   # (195, 39)
        years = list(map(str, range(1980, 2014)))
```

- 💡 이번 CSV는 첫 열에 **`Unnamed: 0`**(저장 시 생긴 불필요한 index 열)이 있어 DV_05/06(열 38개)보다 열이 하나 많음(39개). 분석에는 영향 없지만 `groupby().sum()` 결과에 이 열도 합산돼 나옴(의미 없는 값).

B-2. Pie Chart

Step 1: **`groupby`** (셀 17~19) — Split → Apply → Combine

1. Split: 어떤 기준으로 데이터를 그룹으로 나눔
2. Apply: 각 그룹에 함수 적용 — `.sum()` `.count()` `.mean()` `.std()` `.aggregate()` `.apply()` 등
3. Combine: 결과를 하나의 자료구조로 합침

```
        df_continents = df_can.groupby('Continent', axis=0).sum()   # 대륙별로 묶어 합계
        print(type(df_can.groupby('Continent', axis=0)))            # <class
'pandas.core.groupby.generic.DataFrameGroupBy'>
        df_continents.head()
```

- 📘 groupby의 결과는 **`groupby`** 객체 — `.sum()` 같은 함수를 적용하기 전에는 더 쓸 수 없음.

- 💡 `axis=0` 은 최신 pandas에서 FutureWarning(deprecated) → 빼도 됨(`groupby('Continent').sum()`).

Step 2: plot — 주요 파라미터 (셀 20~21)

파라미터	의미
`autopct`	각 조각에 숫자(퍼센트) 를 표시하는 문자열 또는 함수. 포맷 문자열이면 `fmt % pct`. 조각 안쪽에 표시. 예: `'%1.1f%%'` = 소수 첫째 자리까지 + %
`startangle`	파이 시작 위치를 x축에서 반시계 방향으로 몇 도 회전할지 (`90`이면 첫 조각이 위쪽에서 시작)
`shadow`	파이 아래 그림자(입체감)

```
        df_continents['Total'].plot(kind='pie',
                                    figsize=(5, 6),
                                    autopct='%1.1f%%',     # 퍼센트 표시
                                    startangle=90,         # 시작 각도 90° (Africa)
                                    shadow=True)           # 그림자
        plt.title('Immigration to Canada by Continent [1980 - 2013]')
        plt.axis('equal')                                   # 파이를 원(circle)처럼 보이게
        plt.legend(labels=df_continents.index, loc='upper left')   # 범례
        plt.show()
```

- 💡 `plt.axis('equal')`: x, y 비율을 같게 → 타원이 아니라 정원.

개선: 글자 겹침 해결 (셀 22~23)

- 📘 개선 방법 4가지
  1. 파이 위의 텍스트 라벨 제거(`labels=None`) 후 **`plt.legend()`** 로 별도 범례
  2. **`pctdistance`** 로 퍼센트를 파이 바깥쪽으로 밀어냄
  3. **`colors`** 로 대륙별 색 지정
  4. **`explode`** 로 가장 작은 세 대륙(Africa, North America, Latin America & Caribbean) 강조

```
        colors_list  = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue', 'lightgreen', 'pink']
        explode_list = [0.1, 0, 0, 0, 0.1, 0.1]   # 각 조각을 밖으로 떼어낼 비율

        df_continents['Total'].plot(kind='pie',
                                    figsize=(10, 6),
                                    autopct='%1.1f%%',
                                    startangle=90,
                                    shadow=True,
                                    labels=None,          # 파이 위 라벨 끄기
                                    pctdistance=1.12,     # 각 조각 중심 ~ 퍼센트 글자 시작 위치 비율(>1
이면 바깥)
                                    #colors=colors_list,  # (노트북에서는 주석 처리됨)
                                    #explode=explode_list # (주석 처리됨)
                                    )
        plt.title('Immigration to Canada by Continent [1980 - 2013]', y=1.12, fontsize=15)  # 퍼센트
가 바깥으로 갔으니 제목도 12% 위로
        plt.axis('equal')
        plt.legend(labels=df_continents.index, loc='upper left', fontsize=7)
        plt.show()
```

- 💡 `pctdistance=1.12` → 제목 `y=1.12`로 같이 올려서 안 겹치게 (노트북 주석: “scale the title up by 12% to match pctdistance”).
- 💡 `explode` 리스트 길이 = 조각 수(6). 0이면 붙어 있음, 0.1이면 반지름의 10%만큼 떨어짐.

연습 (셀 24~26): **2013년** 대륙별 비율 pie

```
        explode_list = [0.0, 0, 0, 0.1, 0.1, 0.2]     # 작은 조각들이 겹치지 않게 값 조정 (play with explode values)

        df_continents['2013'].plot(kind='pie',
                                   figsize=(15, 6),
                                   autopct='%1.1f%%',
                                   startangle=90,
                                   shadow=True,
                                   labels=None,
                                   pctdistance=1.12,
                                   explode=explode_list)
        plt.title('Immigration to Canada by Continent in 2013', y=1.12)
        plt.axis('equal')
        plt.legend(labels=df_continents.index, loc='upper left')
        plt.show()
```

- 💡 열을 `['Total']` → `['2013']` 로만 바꾸면 기간이 바뀜.

B-3. Box Plot

개념 (셀 27~28)

- 📘 데이터 분포를 5개 값으로 표현: minimum, first quartile, median, third quartile, maximum
- 📘 pandas Series/DataFrame의 `plot`에 **`kind='box'`**

일본 이민자 box plot (셀 29~34)

```
        # 행 선택을 [['Japan']] 처럼 대괄호 두 겹 → 결과가 Series가 아니라 DataFrame (describe() 등에 유리)
        df_japan = df_can.loc[['Japan'], years].transpose()
        df_japan.head()

        df_japan.plot(kind='box', figsize=(8, 6))
        plt.title('Box plot of Japanese Immigrants from 1980 - 2013')
        plt.ylabel('Number of Immigrants')
        plt.show()
```

- 📘 한 나라만 뽑아도 DataFrame으로 뽑으라는 이유: `describe()`로 백분위수 확인 가능.
- 📘 그래프에서 읽는 값: min ≈ 200, max ≈ 1300, median ≈ 900 / 연도의 **25%**가 연 ~500명 이하(Q1) / **75%**가 ~1100명 이하(Q3)
- `df_japan.describe()` 실제 값: count 34, mean 814.9, std 337.2, min 198, 25% 529, 50% 902, 75% 1079, max 1284

두 나라 비교: China vs India (셀 35~47)

```
        df_CI = df_can.loc[['China', 'India'], years].transpose()    # 열 = 국가
        df_CI.describe()                                             # 통계 요약

        df_CI.plot(kind='box', figsize=(10, 7))
        plt.title('Box plots of Immigrants from China and India (1980 - 2013)')
        plt.ylabel('Number of Immigrants')
        plt.show()
```

- 📘 해석: 중앙값은 비슷(~20,000), 하지만 중국의 범위가 인도보다 더 퍼져 있음. 인도의 최댓값(36,210)은 중국 최댓값(42,584)보다 약 15% 낮음.
- `describe()` 값: China 25% 5512.75 / 50% 19945 / 75% 31568.5, India 25% 10637.75 / 50% 20235 / 75% 28699.5

가로 box plot (셀 46~47)

```
        df_CI.plot(kind='box', figsize=(10, 7), color='blue', vert=False)   # vert=False → 가로,
color로 색 변경(기본은 빨강)
        plt.title('Box plots of Immigrants from China and India (1980 - 2013)')
        plt.xlabel('Number of Immigrants')        # 가로이므로 값 축 라벨은 xlabel
        plt.show()
```

★ Subplots (셀 48~51) — figure를 나눠 여러 그래프

- 📘 figure(전체 캔버스) 를 subplots로 나눠 각각에 plot. 이때 보통 artist layer 사용.

```
        fig = plt.figure()                                  # figure 생성
        ax = fig.add_subplot(nrows, ncols, plot_number)     # subplot 생성
```

- `nrows`, `ncols` = 그리드의 행/열 수 / `plot_number` = 1부터 시작, 행 방향(가로)으로 증가, 최대 nrows×ncols
- 어느 subplot에 그릴지는 **`plot()`**의 **`ax=`** 파라미터로 지정

```
        fig = plt.figure()
        ax0 = fig.add_subplot(1, 2, 1)     # 1행 2열 중 첫 번째
        ax1 = fig.add_subplot(1, 2, 2)     # 1행 2열 중 두 번째

        # Subplot 1: Box plot
        df_CI.plot(kind='box', color='blue', vert=False, figsize=(20, 6), ax=ax0)
        ax0.set_title('Box Plots of Immigrants from China and India (1980 - 2013)')
        ax0.set_xlabel('Number of Immigrants')
        ax0.set_ylabel('Countries')

        # Subplot 2: Line plot
        df_CI.plot(kind='line', figsize=(20, 6), ax=ax1)
        ax1.set_title('Line Plots of Immigrants from China and India (1980 - 2013)')
        ax1.set_ylabel('Number of Immigrants')
        ax1.set_xlabel('Years')
        plt.show()
```

- 📘 Tip: `nrows, ncols, plot_number`가 모두 10 미만이면 3자리 숫자로 축약 가능 — 백의 자리=nrows, 십의 자리=ncols, 일의 자리=plot_number. `subplot(211) == subplot(2, 1, 1)` (2행 1열의 맨 위 plot).

고급: 상위 15개국 **10년 단위(decade)** box plot (셀 52~66)

```
        # Step 1: 상위 15개국
        df_top15 = df_can.sort_values(['Total'], ascending=False, axis=0).head(15)

        # Step 2: 10년 단위 합계 DataFrame
        years_80s = list(map(str, range(1980, 1990)))
        years_90s = list(map(str, range(1990, 2000)))
        years_00s = list(map(str, range(2000, 2010)))

        df_80s = df_top15.loc[:, years_80s].sum(axis=1)    # 국가별(행별) 합계 → axis=1
        df_90s = df_top15.loc[:, years_90s].sum(axis=1)
        df_00s = df_top15.loc[:, years_00s].sum(axis=1)

        new_df = pd.DataFrame({'1980s': df_80s, '1990s': df_90s, '2000s': df_00s})   # 3개 Series를
합침
        new_df.describe()

        # Step 3: box plot
        new_df.plot(kind='box', figsize=(10, 6))
        plt.title('Immigration from top 15 countries for decades 80s, 90s and 2000s')
        plt.show()
```

- 💡 `sum(axis=1)` = 행(국가)별로 열(연도)을 가로로 합산 / `sum(axis=0)` = 열별 합(연도별 전체 합).
- 💡 이 plot의 x축 = 10년 구간(범주), y = 그 구간의 국가별 합계(분포).

★ 이상치(Outlier) 판단 규칙 (셀 65~66)

- 📘 box plot은 데이터를 훑어 이상치를 자동 식별. 이상치 조건:
  - Q3보다 1.5×IQR 이상 큰 값, 또는
  - Q1보다 1.5×IQR 이상 작은 값
- 📘 예 (2000s): Q1 = 36,101.5 / Q3 = 105,505.5 / IQR = Q3 − Q1 = 69,404 → 이상치 기준 > 105,505.5 + 1.5×69,404 = 209,611.5 → China와 India가 이상치(2000년대 합계가 209,611.5 초과)
- 📘 box plot은 고급 시각화 도구로 옵션이 많음(Matplotlib 문서 참고).
- ➕ (DV_04에 추가해 둔 설명과 일치) 일반 공식: `하한 = Q1 − 1.5×IQR`, `상한 = Q3 + 1.5×IQR`, 이를 벗어나면 이상치 점.

B-4. Scatter Plot

데이터 준비 (셀 67~69): 연도별 **전체 국가 합계**

```
        df_tot = pd.DataFrame(df_can[years].sum(axis=0))    # 열(연도)별 합계 → 연도별 총 이민자
(Series → DataFrame)
        df_tot.index = map(int, df_tot.index)               # 연도를 int로 (나중에 회귀에 유용)
        df_tot.reset_index(inplace=True)                    # index(연도)를 일반 열로 되돌림
        df_tot.columns = ['year', 'total']                  # 열 이름 변경
        df_tot.head()                                       # 1980 99137 / 1981 110563 / 1982
104271 / ...
```

- 💡 scatter는 x, y가 모두 "열"이어야 하므로 index였던 연도를 `reset_index`로 열로 꺼냄.

plot (셀 70~72)

```
        df_tot.plot(kind='scatter', x='year', y='total', figsize=(10, 6), color='darkblue')
        plt.title('Total Immigration to Canada from 1980 - 2013')
        plt.xlabel('Year')
        plt.ylabel('Number of Immigrants')
        plt.show()
```

- 📘 **`kind='scatter'`** + **`x=`**, **`y=`**로 x/y에 올 열 이름을 지정해야 함 (line/bar와 다른 점!).
- 📘 scatter는 점을 연결하지 않음. 연도가 갈수록 총 이민자 수 증가하는 상승 추세 → 회귀선(line of best fit) 으로 수학적 분석 가능.

회귀선과 예측 (셀 73~77)

```
        x = df_tot['year']
        y = df_tot['total']
        fit = np.polyfit(x, y, deg=1)      # 최적 다항식 계수 → array([ 5567.09228, -10926195.2 ])
```

`np.polyfit` 인자	의미
`x`	x 좌표
`y`	y 좌표
`deg`	다항식 차수: 1 = 선형, 2 = 이차 …

- 📘 출력은 높은 차수부터 나열된 계수 배열. 선형 y = a·x + b 이므로 [0] = 기울기(slope), [1] = 절편(intercept)

```
        df_tot.plot(kind='scatter', x='year', y='total', figsize=(10, 6), color='darkblue')
        plt.title('Total Immigration to Canada from 1980 - 2013')
        plt.xlabel('Year'); plt.ylabel('Number of Immigrants')

        plt.plot(x, fit[0] * x + fit[1], color='red')                      # 회귀선
        plt.annotate('y={0:.0f} x + {1:.0f}'.format(fit[0], fit[1]), xy=(2000, 150000))   # 식 표시
        plt.show()

        'No. Immigrants = {0:.0f} * Year + {1:.0f}'.format(fit[0], fit[1])  # 'No. Immigrants =
5567 * Year + -10926195'
```

- 📘 2015년 예측: 5567 × 2015 − 10,926,195 = 291,310명. 실제 CIC 2016 보고서 271,845명 → 예측값이 실제와 약 7% 이내(UN 데이터가 CIC와 다소 다를 수 있음을 감안하면 양호).
- 💡 `'{0:.0f}'.format(값)` = 소수점 0자리(정수)로 서식.

연습 (셀 78~84): 덴마크+노르웨이+스웨덴 **합계** scatter

```
        df_countries = df_can.loc[['Denmark', 'Norway', 'Sweden'], years].transpose()
        df_total = pd.DataFrame(df_countries.sum(axis=1))      # 행(연도)별로 세 나라 합산
        df_total.reset_index(inplace=True)
        df_total.columns = ['year', 'total']
        df_total['year'] = df_total['year'].astype(int)        # 연도 문자열 → int

        df_total.plot(kind='scatter', x='year', y='total', figsize=(10, 6), color='darkblue')
        plt.title('Immigration from Denmark, Norway, and Sweden to Canada from 1980 - 2013')
```

```
        plt.xlabel('Year'); plt.ylabel('Number of Immigrants')
        plt.show()
```

B-5. Bubble Plot

개념 (셀 85)

- 📘 bubble plot = scatter plot의 변형으로 3차원(x, y, z) 데이터를 표시. 점 대신 버블을 쓰고, 버블 크기는 세 번째 변수 `z`(가중치, weight) 가 결정.
- 📘 Matplotlib에서는 `plot()`의 `s` 파라미터에 배열 또는 스칼라(각 점의 가중치)를 넘김.
- 💻 사례: 아르헨티나 대공황(1998~2002) 영향 — 이웃 브라질과 비교. 가중치는 연도별 이민자 수의 정규화(normalized) 값.

Step 1: 데이터 (셀 86~87) — **국가 열**이 되도록 전치 + Year 열 만들기

```
        df_can_t = df_can[years].transpose()          # 행=연도, 열=국가(195개)
        df_can_t.index = map(int, df_can_t.index)     # 연도를 int
        df_can_t.index.name = 'Year'                  # index 이름 → reset_index 시 열 이름이 됨
        df_can_t.reset_index(inplace=True)            # Year를 열로
```

Step 2: 정규화 (셀 88~89) — **Feature scaling(min-max)** 으로 값을 [0, 1]로

```
X' = (X − X_min) / (X_max − X_min)       → 최댓값 = 1, 최솟값 = 0
```

```
        norm_brazil    = (df_can_t['Brazil']    - df_can_t['Brazil'].min())    /
(df_can_t['Brazil'].max()    - df_can_t['Brazil'].min())
        norm_argentina = (df_can_t['Argentina'] - df_can_t['Argentina'].min()) /
(df_can_t['Argentina'].max() - df_can_t['Argentina'].min())
```

- 💡 정규화 이유: 국가마다 이민자 규모가 다르므로 0~1로 맞춰 버블 크기를 비교 가능하게.

Step 3: plot (셀 90~92)

- 📘 두 scatter를 한 그래프에 그리려면 `ax` 파라미터로 첫 plot의 axes를 둘째 plot에 넘김.
- 📘 가중치는 `s` 로 전달. 정규화 값(0~1)은 너무 작아 안 보이므로 ×2000으로 키우고, 최솟값(0)도 보이도록 +10.

```
        # Brazil
        ax0 = df_can_t.plot(kind='scatter',
                            x='Year', y='Brazil',
                            figsize=(14, 8),
                            alpha=0.5,                  # 투명도
                            color='green',
                            s=norm_brazil * 2000 + 10,  # 버블 크기(가중치)
                            xlim=(1975, 2015))          # x축 범위

        # Argentina  (ax=ax0 → 같은 그래프에 겹쳐 그림)
        ax1 = df_can_t.plot(kind='scatter',
                            x='Year', y='Argentina',
```

```
                            alpha=0.5, color="blue",
                            s=norm_argentina * 2000 + 10,
                            ax=ax0)

        ax0.set_ylabel('Number of Immigrants')
        ax0.set_title('Immigration from Brazil and Argentina from 1980 to 2013')
        ax0.legend(['Brazil', 'Argentina'], loc='upper left', fontsize='x-large')
```

- 📘 해석: 버블이 클수록 그 해 이민자 수가 (1980~2013 데이터 대비) 더 많음. 아르헨티나는 **1998~2002 대공황** 때 이민 증가, 1985~1993 에도 유사한 급증(1974~1990 대공황). 브라질은 1999년 헤알화 ~35% 하락 (Samba Effect) 과 긴축 프로그램 후 2010년 급증.

연습 (셀 93~99): 중국 vs 인도 bubble plot

```
        norm_china = (df_can_t['China'] - df_can_t['China'].min()) / (df_can_t['China'].max() -
df_can_t['China'].min())
        norm_india = (df_can_t['India'] - df_can_t['India'].min()) / (df_can_t['India'].max() -
df_can_t['India'].min())

        ax0 = df_can_t.plot(kind='scatter', x='Year', y='China', figsize=(14, 8),
                            alpha=0.5, color='green', s=norm_china * 2000 + 10, xlim=(1975, 2015))
        ax1 = df_can_t.plot(kind='scatter', x='Year', y='India',
                            alpha=0.5, color='blue', s=norm_india * 2000 + 10, ax=ax0)
        ax0.set_ylabel('Number of Immigrants')
        ax0.set_title('Immigration from China and India from 1980 - 2013')
        ax0.legend(['China', 'India'], loc='upper left', fontsize='x-large')
```

PART C. 정리표

C-1. plot별 사용 변수·데이터

Plot	데이터 준비	x	y	(z)	핵심 코드
Pie	`groupby('Continent').sum()`	— (조각 = 대륙)	`['Total']` 또는 `['2013']` 값 → 비율	—	`plot(kind='pie', autopct=, startangle=, shadow=, labels=None, pctdistance=, colors=, explode=)`
Box (1국가)	`.loc[['Japan'], years].transpose()`	열(국가)	연도별 값 분포	—	`plot(kind='box')`
Box (여러 국가)	`.loc[['China','India'], years].transpose()`	열별 상자	값	—	`plot(kind='box', vert=False, color=)`
Box (decade)	top15 → decade별 `sum(axis=1)` → `new_df`	1980s/1990s/2000s	국가별 10년 합계	—	`new_df.plot(kind='box')`
Scatter	`df[years].sum(axis=0)` → `reset_index` → `['year','total']`	`year`	`total`	—	`plot(kind='scatter', x='year', y='total')`
Bubble	`df[years].transpose()` + `Year` 열 + 정규화	`Year`	국가 열 (`'Brazil'`)	`s` = 정규화값 ×2000+10	`plot(kind='scatter', ..., s=, alpha=, ax=)`

C-2. 파라미터 사전 (DV_07 신규)

파라미터	적용	의미
`autopct='%1.1f%%'`	pie	조각 안에 퍼센트 표시
`startangle`	pie	시작 각도(반시계)
`shadow`	pie	그림자
`labels=None`	pie	파이 위 라벨 끄기 (→ `plt.legend`)
`pctdistance`	pie	퍼센트 글자 위치(중심 대비 비율)
`colors`	pie	조각 색 리스트
`explode`	pie	조각 떼어내기 비율 리스트
`vert=False`	box	가로 상자
`color`	box, scatter	색
`x=`, `y=`	scatter	열 이름으로 축 지정
`s`	scatter(bubble)	점(버블) 크기 — 스칼라 또는 배열

파라미터	적용	의미
`alpha`	scatter	투명도
`xlim=(min,max)`	scatter 등	x축 범위
`ax=`	전부	어느 axes에 그릴지 (subplot 배치, 겹쳐 그리기)

C-3. 시험 대비 포인트

1. Pie: `groupby().sum()` 필요 / `autopct`, `explode`, `startangle`, `pctdistance`, `labels=None`+`legend` 의미 / `plt.axis('equal')` = 원형 유지.
2. groupby = Split → Apply → Combine, groupby 객체는 함수를 적용해야 결과가 나옴.
3. Box: 5개 숫자(min, Q1, median, Q3, max) / IQR = Q3−Q1 / 이상치 = Q3+1.5·IQR 초과 또는 Q1−1.5·IQR 미만 / `describe()`로 수치 확인 / `vert=False`는 가로.
4. 한 나라만 box plot할 때는 `.loc[['Japan'], ...]` (대괄호 두 겹 → DataFrame).
5. Subplot: `fig.add_subplot(nrows, ncols, plot_number)`, `plot_number`는 1부터 행 방향 증가, `plot(..., ax=ax0)`, 축약형 `subplot(211) == subplot(2,1,1)`.
6. Scatter는 `x=`, `y=` 열 이름 지정이 필수. 연도가 index면 `reset_index()`로 열로.
7. 회귀선: `np.polyfit(x, y, deg=1)` → `[기울기, 절편]` → `plt.plot(x, fit[0]*x + fit[1])`.
8. Bubble = scatter + 세 번째 변수(`s`). 정규화(min-max)로 [0,1] → `×2000 + 10`.
9. 두 plot 겹치기: 첫 plot의 반환값 `ax0` → 두번째 `plot(..., ax=ax0)`.
10. `sum(axis=0)` = 열별(위→아래) 합, `sum(axis=1)` = 행별(좌→우) 합.

DV_08 — Waffle Chart · Word Cloud · Seaborn(Categorical / Regression)

> 표기: 📘 = 슬라이드 / 💻 = Colab 실습 노트북 / 💡 = 이해용 주석 / ➕ = 추가 설명 자료: `DV_08_waffle_wordcloud_regression.pdf` + `DV_08_code_waffle_wordcloud_regression.ipynb`

PART A. 이론 (슬라이드)

A-1. Waffle Chart (슬라이드 2~4)

- 📘 범주형 데이터를 정사각형 타일(square tiles) / 셀(cells) 형태로 표현
- 📘 서로 다른 범주의 비율(proportion) 또는 퍼센트 표시
- 📘 모든 유형의 청중에게 데이터를 단순화
- 📘 용도 7가지: 시장 점유율 분석(market share) / 인구통계 표현(demographic) / 프로젝트 진행 추적(작업·마일스톤 완료 상태) / 예산 배분(budget allocation) / 설문 응답 요약(survey responses) / 선거 결과(투표 결과를 명확히) / 제품 판매 분석(product sales)
- 슬라이드 4: PyWaffle 라이브러리 예시 코드

```
        import matplotlib.pyplot as plt
        from pywaffle import Waffle

        # waffle chart용 데이터 생성
        data = df_dsn[['Africa', 'Asia', 'Europe', 'Oceania']]      # (슬라이드 예시 변수)

        # waffle chart figure 설정
        fig = plt.figure(
            FigureClass=Waffle,
            rows=20,
            columns=20,
            values=data,
            legend={'labels': ['Africa', 'Asia', 'Europe', 'Oceania'],
                    'loc': 'upper left', 'bbox_to_anchor': (1, 1)})

        plt.show()
```

A-2. Word Cloud (슬라이드 5~6)

- 📘 인기 있는 데이터 시각화 방법 / 텍스트 내용의 간결한 요약 제시 / 단어의 중요도를 표현 (슬라이드 그림: 긴 글 → 자주 나온 단어가 크게 보이는 구름)
- 📘 용도 5가지: 소셜 미디어 분석(인기 주제·감성 추출) / 고객 피드백 분석(리뷰 요약) / 콘텐츠 분석(텍스트 분석) / 시장 조사(설문 응답 분석) / 이력서·채용공고 분석(핵심 기술·키워드 강조)

A-3. Seaborn (슬라이드 7)

- 📘 Matplotlib 기반의 Python 시각화 라이브러리
- 📘 내장 테마와 색 팔레트로 plot 시각 품질 향상
- 📘 Matplotlib으로 ~20줄 필요한 시각화를 Seaborn은 5배 적은 줄로 가능
- 📘 특수 plot: Regression plot / Distribution plot / Categorical plot / 회귀선이 있는 scatter plot 생성

A-4. Regression Plot (슬라이드 8~11)

- 슬라이드 8~9 (df_total: Year, Total 열 — 1980=99137, 1981=110563 …)

```
        import seaborn as sns
        ax = sns.regplot(x='year', y='total', data=df_tot)                          # 기본 (주황/빨강)
        ax = sns.regplot(x='year', y='total', data=df_tot, color='green', marker='+')   # 색과 마커 변경
```

- 📘 CI(신뢰구간) 슬라이드 10
  - 초록 음영 영역 = 회귀선 주변의 신뢰구간(confidence interval, CI)
  - 같은 모집단에서 새 샘플을 뽑았을 때 회귀선이 얼마나 달라질 수 있는지 보여줌
  - 신뢰수준(90%, 95%, 99%) 이 음영 폭을 결정
  - 📘 "90% 신뢰구간 = 참(true) 회귀선이 이 범위 안에 있다고 90% 확신한다"
- 슬라이드 11: 95% / 90% / 99% Confidence Interval 비교 (speed vs dist) — 💡 신뢰수준이 높을수록(99%) 음영이 더 넓고, 낮을수록(90%) 좁음.

A-5. Categorical Plots (슬라이드 12~13)

- 슬라이드 12: `sns.countplot(x='Continent', data=df_canada)` — 대륙별 개수(Asia, Europe, Africa, Oceania, LA, HK, Macao 막대; y축 = count)
- 슬라이드 13: `sns.barplot(x='Continent', y='Total', data=df_canada)` — 대륙별 Total (막대 위의 검은 세로선 = 신뢰구간 오차 막대)

A-6. Recap (슬라이드 14)

- Waffle chart: 범주형 데이터를 정사각형 타일/셀로 표현하는 기법. 여러 분야에서 사용 가능
- Word cloud: 텍스트 데이터를 시각적으로 흥미롭고 유익하게 제시. 여러 분야에서 사용
- Seaborn: Matplotlib 기반. 통계 그래픽을 그리기 위한 고수준(high-level) 인터페이스
- Scatter plot + regression line을 Seaborn으로 한 줄로 생성
- 💡 슬라이드 15의 Lab 안내에 "Pie, Box, Scatter"라고 적혀 있으나 DV_07 슬라이드를 복사한 오기로 보임. 실제 Lab = waffle, wordcloud, regression.

PART B. Colab 실습 (노트북 `DV_08_code_waffle_wordcloud_regression.ipynb`)

B-0. 전체 순서

```
[준비] matplotlib/mpatches, numpy, pandas, PIL.Image, !pip install seaborn wordcloud, import seaborn as sns, wordcloud
[데이터] read_csv → set_index('Country')
[Waffle]  (직접 구현 7단계) 비율 → 크기 → 타일 수 → 행렬 채움 → matshow → 격자 → 범례   ⇒ 함수화 ⇒ PyWaffle(Waffle)
[WordCloud] WordCloud, STOPWORDS → 텍스트 읽기 → stopwords → WordCloud().generate → imshow → mask → 국가명 반복 텍스트
[Seaborn] countplot → barplot(ci) → regplot(color, marker, scatter_kws, ax.set, set_style, set(font_scale))
```

B-1. 준비 (셀 2~10)

```
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches       # waffle chart(범례 사각형)에 필요
mpl.style.use('ggplot')

import numpy as np
import pandas as pd
from PIL import Image                        # 이미지를 배열로 변환 (word cloud mask용)

!pip install seaborn wordcloud
import seaborn as sns
import wordcloud

print('Matplotlib version: ', mpl.__version__)
print('Seaborn version: ', sns.__version__)          # 0.13.2
print('WordCloud version: ', wordcloud.__version__)  # 1.9.4

df_can = pd.read_csv('DV_05_data_canada_immigration.csv')
print(df_can.shape)                     # (195, 40)
df_can.set_index('Country', inplace=True)
```

- 💡 `!pip install ...` = Colab에서 셀 명령으로 패키지 설치(`!`로 시작).

B-2. Waffle Chart

- 📘 노트북 설명: waffle chart = 목표 대비 진행 상황을 보여주는 데 흔히 쓰임. 셀 위주 시각물(예: Excel 대시보드)에 효과적.
- 📘 waffle chart는 Python 시각화 라이브러리에 기본 내장되어 있지 않음 → 처음부터 직접 만든 뒤, 마지막에 PyWaffle 패키지로 같은 것을 생성.
- 사례: 덴마크·노르웨이·스웨덴의 Total 이민자 비율.

```
df_dsn = df_can.loc[['Denmark', 'Norway', 'Sweden'], :]     # 3개국, 모든 열
```

Step 1. 각 범주의 전체 대비 **비율**

```
total_values = df_dsn['Total'].sum()
category_proportions = df_dsn['Total'] / total_values
# Denmark 0.322557, Norway 0.192409, Sweden 0.485034
```

Step 2. waffle chart **전체 크기**

```
width = 40       # 가로 타일 수
height = 10      # 세로 타일 수
total_num_tiles = width * height     # 400
```

Step 3. 비율 → **범주별 타일 수**

```
tiles_per_category = (category_proportions * total_num_tiles).round().astype(int)
# Denmark 129, Norway 77, Sweden 194
```

Step 4. waffle 모양의 **행렬(matrix)** 만들고 채우기

```
waffle_chart = np.zeros((height, width), dtype=np.uint)    # 0으로 채운 (10×40) 행렬

category_index = 0
tile_index = 0
for col in range(width):                  # 열 방향(왼→오)으로 먼저
    for row in range(height):             # 각 열을 위→아래로
        tile_index += 1
        # 현재 범주에 할당된 타일 수를 다 채웠으면 다음 범주로
        if tile_index > sum(tiles_per_category[0:category_index]):
            category_index += 1
        waffle_chart[row, col] = category_index     # 범주 번호(1,2,3)를 저장
```

- 💡 행렬 값 = 범주 번호(Denmark=1, Norway=2, Sweden=3). 열 단위로 채우므로 같은 범주가 세로 줄 단위로 연속.
- 💡 `tile_index > sum(앞 범주들의 타일 수)` 일 때 `category_index += 1`.

Step 5. 행렬을 **그림으로** (**`matshow`**)

```
fig = plt.figure()
colormap = plt.cm.coolwarm
plt.matshow(waffle_chart, cmap=colormap)     # 행렬 값을 색으로 표시
plt.colorbar()
plt.show()
```

Step 6. 꾸미기 — 격자(타일 경계)

```
ax = plt.gca()                                              # 현재 axes 가져오기
ax.set_xticks(np.arange(-.5, (width), 1), minor=True)       # 보조 눈금(타일 경계 위치)
```

```
        ax.set_yticks(np.arange(-.5, (height), 1), minor=True)
        ax.grid(which='minor', color='w', linestyle='-', linewidth=2)   # 보조 눈금 기준 흰색 격자선
        plt.xticks([])     # 일반 눈금 숫자 제거
        plt.yticks([])
```

- 💡 눈금을 −0.5, 0.5, 1.5⋯ 로 두는 이유: 타일의 중심이 정수 좌표이므로 경계는 .5 지점.

Step 7. **범례** 추가

```
        values_cumsum = np.cumsum(df_dsn['Total'])            # 누적합 (색 매칭용)
        total_values = values_cumsum[len(values_cumsum) - 1]  # 전체 합 (마지막 누적값)

        legend_handles = []
        for i, category in enumerate(df_dsn.index.values):
            label_str = category + ' (' + str(df_dsn['Total'][i]) + ')'        # 예: 'Denmark
(3901)'
            color_val = colormap(float(values_cumsum[i]) / total_values)         # 차트의 색과 같은 색
            legend_handles.append(mpatches.Patch(color=color_val, label=label_str))

        plt.legend(handles=legend_handles, loc='lower center',
                   ncol=len(df_dsn.index.values),                    # 범례 열 수 = 범주 수
                   bbox_to_anchor=(0., -0.2, 0.95, .1))
```

- 💡 누적합(**`np.cumsum`**)을 쓰는 이유: 차트의 각 범주 색은 `coolwarm` 컬러맵에서 (범주 번호/전체) 위치 값으로 결정 → 범례 색도 같은 위치 값으로 맞춤. (`mpatches.Patch` = 색 사각형 범례 항목)

7단계를 함수로: **`create_waffle_chart`** (셀 33~38)

파라미터	의미
`categories`	범주 이름들
`values`	범주에 대응하는 값
`height`	waffle 높이
`width`	waffle 너비
`colormap`	컬러맵 클래스
`value_sign`	값에 붙는 기호(%, $ 등). 기본값 빈 문자열. `'%'`면 값 뒤에, 그 외 기호는 값 앞에 붙임

```
        width = 40; height = 10
        categories = df_dsn.index.values
        values = df_dsn['Total']
        colormap = plt.cm.coolwarm
        create_waffle_chart(categories, values, height, width, colormap)
        # Total number of tiles is 400 / Denmark: 129 / Norway: 77 / Sweden: 194
```

- 💡 함수 안에서 타일 수 출력 시 `df_dsn.index.values[i]`를 직접 사용 → (코드상) 이 함수는 `df_dsn`에 의존. 실습 시 다른 데이터에 쓰려면 이 부분 주의.

PyWaffle로 한 번에 (셀 39~41) ⭐

```
        !pip install pywaffle
        from pywaffle import Waffle

        fig = plt.figure(FigureClass=Waffle,                # figure 클래스를 Waffle로 지정
                         rows=10, columns=40,               # 행/열 개수
                         values=df_dsn['Total'],            # 표시할 데이터
                         cmap_name='tab20',                 # 컬러 스킴
                         legend={'labels': [f"{k} ({v})" for k, v in zip(df_dsn.index.values,
df_dsn.Total)],
                                 'loc': 'lower left', 'bbox_to_anchor': (0, -0.1), 'ncol': 3})
        plt.show()
```

- 💡 직접 구현 ↔ PyWaffle 대응: 비율·타일 수 계산·행렬 채움·격자·범례를 PyWaffle이 알아서 처리. `FigureClass=Waffle`이 핵심.
- 📘 범례 라벨은 리스트 컴프리헨션(list comprehension) 으로 index와 Total에서 생성 (노트북 주석).
- 📘 PyWaffle 파라미터: `rows`, `columns`, `values`, `cmap_name`, `legend`(= `labels`, `loc`, `bbox_to_anchor`, `ncol`)

연습: 중국·인도 Total 비율 waffle

```
        df_dsn = df_can.loc[['China', 'India'], :]
        fig = plt.figure(FigureClass=Waffle,
                         rows=20, columns=30,
                         values=df_dsn['Total'],
                         cmap_name='tab20',
                         legend={'labels': [f"{k} ({v})" for k, v in zip(df_dsn.index.values,
df_dsn.Total)],
                                 'loc': 'lower left', 'bbox_to_anchor': (0, -0.1), 'ncol': 2})   #
범주 2개 → ncol=2
        plt.show()
```

B-3. Word Cloud

- 📘 word cloud(= text cloud, tag cloud): 텍스트 자료(연설, 블로그, 데이터베이스 등)에서 특정 단어가 많이 나올수록 더 크고 굵게 표시.
- 📘 파이썬 패키지 **`word_cloud`** — Andreas Mueller 개발.

```
        from wordcloud import WordCloud, STOPWORDS      # WordCloud 클래스와 불용어 집합
```

- 💻 텍스트 예시: Lewis Carroll의 *Alice's Adventures in Wonderland*

단계별 코드

```
        # 1) 텍스트 파일 읽기
        with open('DV_08_data_alice_novel.txt', 'r', encoding='utf-8') as file:
            alice_novel = file.read()
        print(alice_novel[:500])

        # 2) 불용어(stopwords) 집합 — set()으로 중복 제거
        stopwords = set(STOPWORDS)
```

```
        # 3) word cloud 객체 생성 → 텍스트로 생성
        alice_wc = WordCloud()
        alice_wc.generate(alice_novel)

        # 4) 표시
        plt.imshow(alice_wc, interpolation='bilinear')    # 이미지처럼 표시
        plt.axis('off')                                   # 축 숨김
        plt.show()
```

- 📘 결과: 가장 흔한 단어는 Alice, said, little, Queen 등.
- 📘 크게 보려면 `plt.figure(figsize=(14, 18))` 로 그림 크기를 키움 (덜 흔한 단어도 보임).
- 💡 `interpolation='bilinear'`: 이미지 확대 시 부드럽게 보간. `plt.axis('off')`: 좌표축 제거(그림처럼 보이게).

불용어 추가

- 📘 `said`는 별 의미가 없는 단어 → stopwords에 추가 후 재생성

```
        stopwords.add('said')                             # 불용어 추가
        alice_wc = WordCloud(stopwords=stopwords)         # 불용어 지정
        alice_wc.generate(alice_novel)
```

마스크(mask)로 모양 만들기 (셀 62~67)

- 📘 단어를 임의 모양(mask) 위에 입힐 수 있음 (앨리스와 토끼 모양).

```
        alice_mask = np.array(Image.open('DV_08_data_alice_mask.png'))     # 이미지 → numpy 배열

        alice_wc = WordCloud(background_color='white',      # 배경색
                             max_words=2000,                # 최대 단어 수
                             mask=alice_mask,               # 모양 마스크
                             stopwords=stopwords)           # 불용어
        alice_wc.generate(alice_novel)
        fig = plt.figure(figsize=(14, 18))
        plt.imshow(alice_wc, interpolation='bilinear')
        plt.axis('off')
        plt.show()
```

WordCloud() 파라미터	의미
`stopwords`	제외할 단어 집합
`background_color`	배경 색
`max_words`	그릴 최대 단어 수
`mask`	모양(이미지 배열)

- ➕ `max_words`의 기본값은 200 (이 노트북은 마스크 예제에서 2000으로 지정).

이민 데이터로 word cloud 만들기 (셀 69~79)

- 📘 이민 데이터는 텍스트가 없음 → 국가 이름을 "기여도 비율만큼 반복"한 문자열을 만들어 사용 (총 90단어 기준).

```
        total_immigration = df_can['Total'].sum()        # 6409153 (1980~2013 전체 이민자)

        max_words = 90
        word_string = ''
        for country in df_can.index.values:
            if country.count(" ") == 0:        # 띄어쓰기 없는 한 단어 국가명만 (단어 하나로 취급하려고)
                repeat_num_times = int(df_can.loc[country, 'Total'] / total_immigration *
max_words)
                word_string = word_string + ((country + ' ') * repeat_num_times)    # 국가명을 비율만
큼 반복

        wordcloud = WordCloud(background_color='white').generate(word_string)
        plt.figure(figsize=(14, 18))
        plt.imshow(wordcloud, interpolation='bilinear')
        plt.axis('off')
        plt.show()
```

- 📘 불용어 불필요(국가명만 있으므로). 결과: 이민자 대부분이 15개국 출신 (China, India, Philippines, Pakistan 등이 크게 표시).
- 💡 word cloud = "단어 빈도(반복 횟수)"가 글자 크기. 숫자 데이터를 word cloud로 만들려면 값에 비례해 단어를 반복시키면 됨.
- 📘 아이디어: 캐나다 지도를 mask로 써서 위에 word cloud 얹기.

B-4. Seaborn

- 📘 노트북 설명: 이전 DV_07에서 scatter + 회귀선을 만드는 데 ~20줄이 필요했음 → seaborn으로 얼마나 효율적인지 확인.

Categorical plot — `countplot` (셀 82~89)

```
        df_can['Continent'].unique()        # ['Asia','Europe','Africa','Oceania','Latin America
and the Caribbean','Northern America']
        sns.countplot(x='Continent', data=df_can)      # 대륙별 "행 개수"(= 나라 수)
```

- 📘 countplot = 범주형 변수에 대한 히스토그램(양적 변수 대신 범주형 변수).
- 문제: x축 라벨이 길어 겹침 → 이름을 줄이고 figure 키움

```
        df_can1 = df_can.replace('Latin America and the Caribbean', 'L-America')
        df_can1 = df_can1.replace('Northern America', 'N-America')
        plt.figure(figsize=(15, 10))
        sns.countplot(x='Continent', data=df_can1)
```

- 💡 `df.replace(old, new)`: 값 치환.

Categorical plot — `barplot` (셀 90~95)

- 📘 범주형 변수로 groupby를 수행하고 집계 값(기본 평균)을 막대로, 신뢰구간(CI)과 함께 표시.

```
        plt.figure(figsize=(15, 10))
        sns.barplot(x='Continent', y='Total', data=df_can1)     # 대륙별 Total의 평균 + 오차 막대(CI)

        df_Can2 = df_can1.groupby('Continent')['Total'].mean()    # 값 검증 (평균과 일치)
        # Africa 11462.0 / Asia 67710.08 / Europe 32812.72 / L-America 23186.30 / N-America
120571.0 / Oceania 3941.0
```

- 📘 CI를 90%로 변경: `ci=90` (⚠ 최신 seaborn에서 deprecated → `errorbar=('ci', 90)` 로 대체, 노트북 출력 경고)

```
        sns.barplot(x='Continent', y='Total', data=df_can1, ci=90)       # (구 방식)
        # sns.barplot(..., errorbar=('ci', 90))                           # (신 방식)
```

- 💡 barplot의 막대 높이 = 합계가 아니라 평균(mean). pandas `plot(kind='bar')`와 다름.
- 💡 countplot vs barplot: countplot은 y를 지정하지 않음(개수를 셈) / barplot은 y 변수의 집계값(평균) 을 그림.

Regression plot — **`regplot`** (셀 96~114)

- 📘 seaborn에서는 **`regplot`** 함수 호출만으로 회귀 plot 생성.

```
        years = list(map(str, range(1980, 2014)))
        df_tot = pd.DataFrame(df_can[years].sum(axis=0))      # 연도별 총 이민자
        df_tot.index = map(float, df_tot.index)               # 연도를 float으로 (회귀용)
        df_tot.reset_index(inplace=True)
        df_tot.columns = ['year', 'total']
```

- ⚠ DV_07은 `int`, 이번엔 `float`으로 변환 (둘 다 회귀 가능).

점진적으로 꾸미기 (한 줄 → 5줄)

```
        sns.regplot(x='year', y='total', data=df_tot, ci=95)                                    # ①
기본: scatter + 회귀선 + 95% CI 음영
        sns.regplot(x='year', y='total', data=df_tot, ci=95, color='green')                     # ②
색 변경
        ax = sns.regplot(x='year', y='total', data=df_tot, ci=95, color='green', marker='+')  # ③
마커 모양 '+'

        plt.figure(figsize=(15, 10))                                                            # ④
크기 키우기
        sns.regplot(..., marker='+')

        plt.figure(figsize=(15, 10))                                                            # ⑤
마커 크기 + 제목/라벨
        ax = sns.regplot(x='year', y='total', data=df_tot, ci=95, color='green', marker='+',
                         scatter_kws={'s': 200})                                # 점(scatter) 옵션 딕셔너
리: 크기 s=200
        ax.set(xlabel='Year', ylabel='Total Immigration')                       # x/y 라벨 (한 번에 설정)
        ax.set_title('Total Immigration to Canada from 1980 - 2013')            # 제목

        plt.figure(figsize=(15, 10))                                                            # ⑥
글자 크기
        sns.set(font_scale=1.5)                                                 # 눈금, 제목, 라벨 글자 크기
```

```
1.5배
        ax = sns.regplot(...)
```

배경 스타일 변경

```
        sns.set_style('ticks')        # 흰 배경 (격자 없음, 눈금 표시)
        sns.set_style('whitegrid')    # 흰 배경 + 격자선
```

regplot 파라미터	의미
`x=`, `y=`, `data=`	열 이름과 DataFrame (DataFrame 친화적)
`ci`	신뢰구간 수준 (기본 95). 음영 폭
`color`	점과 회귀선 색
`marker`	점 모양 (`'+'`, `'o'` …)
`scatter_kws={'s': 200}`	scatter(점)에 전달할 옵션 딕셔너리 (`s` = 점 크기)

- 💡 `ax.set(xlabel=..., ylabel=...)` 는 seaborn plot이 반환한 Axes 객체(**ax**) 에 설정 (DV_06의 artist layer 방식).
- 📘 결론: “scatter + 회귀선 + 꾸미기를 5줄로.”

연습: 덴마크·스웨덴·노르웨이 합계 regplot

```
        df_countries = df_can.loc[['Denmark', 'Norway', 'Sweden'], years].transpose()
        df_total = pd.DataFrame(df_countries.sum(axis=1))
        df_total.reset_index(inplace=True)
        df_total.columns = ['year', 'total']
        df_total['year'] = df_total['year'].astype(int)

        plt.figure(figsize=(15, 10))
        sns.set(font_scale=1.5)
        sns.set_style('whitegrid')
        ax = sns.regplot(x='year', y='total', data=df_total, ci=95, color='green', marker='+',
scatter_kws={'s': 200})
        ax.set(xlabel='Year', ylabel='Total Immigration')
        ax.set_title('Total Immigration from Denmark, Sweden, and Norway to Canada from 1980 -
2013')
```

PART C. 정리표

C-1. plot별 사용 변수·함수

Plot	라이브러리	데이터	핵심 코드	주요 파라미터
Waffle (직접)	matplotlib	범주별 `Total` → 비율 → 타일 수 → 행렬	`np.zeros`, 이중 for, `plt.matshow`, `mpatches.Patch`, `plt.legend`	`width`, `height`, `colormap`
Waffle (PyWaffle)	pywaffle	`values=df['Total']`	`plt.figure(FigureClass=Waffle, ...)`	`rows`, `columns`, `values`, `cmap_name`, `legend`
Word cloud	wordcloud	텍스트(문자열)	`WordCloud(...).generate(text)` → `plt.imshow`	`stopwords`, `background_color`, `max_words`, `mask`
countplot	seaborn	범주형 열 (y 없음)	`sns.countplot(x='Continent', data=df)`	—
barplot	seaborn	범주형 x + 수치 y	`sns.barplot(x=, y=, data=)`	`ci`/`errorbar` (CI), 막대 = 평균
regplot	seaborn	수치 x + 수치 y (열 이름)	`sns.regplot(x='year', y='total', data=df_tot)`	`ci`, `color`, `marker`, `scatter_kws`

C-2. 시험 대비 포인트

1. Waffle chart = 범주형 데이터의 비율을 타일로. 용도(시장 점유율, 예산, 설문, 선거 등).
2. Waffle 직접 구현 7단계: ① 비율 ② 전체 크기 ③ 타일 수 ④ 행렬 채우기 ⑤ `matshow` ⑥ 격자(minor ticks) ⑦ 범례. → PyWaffle은 **`FigureClass=Waffle`** 로 대체.
3. Word cloud: 단어 빈도가 클수록 크게. `stopwords`(불용어), `mask`(모양), `max_words`. 숫자 데이터는 반복 텍스트로 변환.
4. `plt.imshow(wc, interpolation='bilinear')` + `plt.axis('off')`.
5. Seaborn: Matplotlib 기반 / 짧은 코드(~20줄→5배 적게) / 테마·팔레트 / 통계 plot(regression, distribution, categorical).
6. countplot: 범주별 개수(y 없음). barplot: 범주별 평균 + CI 오차 막대.
7. regplot: scatter + 회귀선 + 신뢰구간 음영. `ci` ↑ → 음영 폭 ↑ (90% < 95% < 99%). “90% CI = 참 회귀선이 이 범위에 있다고 90% 확신”.
8. regplot 꾸미기: `color`, `marker`, `scatter_kws={'s':...}`, `ax.set(xlabel=, ylabel=)`, `ax.set_title`, `sns.set(font_scale=)`, `sns.set_style('ticks' | 'whitegrid')`.
9. Seaborn은 **`x='열이름', y='열이름', data=df`** 형태 (pandas DataFrame 친화적, DV_04 Matplotlib vs Seaborn 비교표와 연결).

DV_09 — Folium으로 지도 만들기 (Maps · Markers · Choropleth)

> 표기: 📘 = 슬라이드 / 💻 = Colab 실습 노트북 / 💡 = 이해용 주석 / ➕ = 추가 설명 자료: `DV_09_folium.pdf` + `DV_09_Creating_map_by_folium_1.ipynb` ⭐ 중간고사 범위의 마지막 강의(~DV_09)

PART A. 이론 (슬라이드)

A-1. Folium이란? (슬라이드 2)

- 📘 지리공간(geospatial) 데이터 시각화를 위해 만들어진 강력한 Python 라이브러리
- 📘 위도(latitude)와 경도(longitude) 값으로 세계 어디든 지도를 만들 수 있음. 지도 위에 마커(marker)와 클러스터(cluster) 를 겹쳐 시각화 가능
- 📘 다양한 스타일의 지도 생성 가능: street level map, stamen map 등

A-2. 지도 만들기 (슬라이드 3~6)

슬라이드	코드	내용
3. 세계 지도	`import folium` → `world_map = folium.Map()` → `world_map`	파라미터 없이 기본 세계 지도
4. 캐나다 지도	`folium.Map(location=[56.130, -106.35], zoom_start=4)`	location(중심 좌표) 과 zoom_start(초기 확대 수준) (빨간 박스 강조)
5. Stamen Toner	`folium.Map(location=[56.130, -106.35], zoom_start=4, tiles='Stamen Toner')`	tiles 파라미터로 스타일 지정 — 흑백 고대비
6. Stamen Terrain	`... tiles='Stamen Terrain'`	지형(산, 숲 등) 이 보이는 스타일

- 📘 Recap: tiles 파라미터로 다양한 지도 스타일을 만들 수 있다. **`location`** 파라미터 = 지도 중심점의 위도·경도 좌표.

A-3. 마커가 있는 지도 (슬라이드 7~13)

- 📘 마커는 지도에 상호작용성과 맥락을 더함. 특정 위치/관심 지점을 나타내며 클릭하면 추가 정보 제공. “길 안내 표지판”처럼 중요한 요소를 강조.

① 지도 초기화 (슬라이드 8)

```
import folium
canada_map = folium.Map(location=[56.1304, -106.3468], zoom_start=4)   # 캐나다 중심
canada_map
```

② 마커와 라벨 추가 (슬라이드 9) — 온타리오 주

```
folium.Marker(location=[51.2538, -85.3232], popup='Ontario').add_to(canada_map)
```

- 📘 `popup` = 클릭하면 나오는 라벨. `.add_to(지도)` 로 지도에 추가.

③ 빨간 원 마커 — FeatureGroup (슬라이드 10~11)

```
        canada_map = folium.Map(location=[56.130, -106.35], zoom_start=4)

        # 피처 그룹 생성
        ontario = folium.map.FeatureGroup()

        # 스타일을 가진 CircleMarker를 그룹에 추가
        ontario.add_child(
            folium.features.CircleMarker([51.25, -85.32], radius=5, color='red', fill_color='Red')
        )

        # 피처 그룹을 지도에 추가
        canada_map.add_child(ontario)

        # 라벨 붙이기
        folium.Marker([51.25, -85.32], popup='Ontario').add_to(canada_map)
```

- 💡 FeatureGroup = 여러 요소를 묶는 그룹 레이어. `add_child()` 로 요소를 넣고, 그룹을 지도에 다시 `add_child()` 로 붙임.

④ 여러 마커 (슬라이드 12~13)

```
        # 위치와 팝업 목록
        locations = [
            {"location": [45.4215, -75.6989],  "popup": "Ottawa"},
            {"location": [53.5461, -113.4938], "popup": "Edmonton"},
            {"location": [49.2827, -123.1207], "popup": "Vancouver"},
            # 필요하면 더 추가
        ]
        # 각 위치마다 마커 추가
        for loc in locations:
            folium.Marker(location=loc["location"], popup=loc["popup"]).add_to(map)
        map          # 마커가 있는 지도 표시
```

⑤ MarkerCluster (슬라이드 13) — 마커를 군집(클러스터) 으로 묶기

```
        from folium.plugins import MarkerCluster

        marker_cluster = MarkerCluster().add_to(map)      # 클러스터 객체 생성 후 지도에 추가
        for loc in locations:
            folium.Marker(location=loc["location"], popup=loc["popup"]).add_to(marker_cluster)   #
지도 대신 cluster에 추가
```

- 📘 Recap: popup 파라미터 = 클릭 시 라벨. 마커는 상호작용성과 맥락을 더함.

A-4. Choropleth Map (슬라이드 14~17)

- 📘 슬라이드 14: 예시 — 세계 영아 사망률 지도, 미국 주별 인구밀도 지도 (값이 클수록 진한 색)
- 📘 Recap: choropleth map = 통계 변수의 측정값에 비례해 지역을 음영/무늬로 표시한 주제도(thematic map)
- 📘 슬라이드 15 — GeoJSON 파일: Folium은 GeoJSON, Pandas DataFrame, NumPy 배열 등 여러 소스로 지도를 만들 수 있음. 샘플 구조:

```
        { "type": "FeatureCollection",
          "features": [{ "type": "Feature",
                         "properties": { "name": "Brunei" },
                         "geometry": { "type": "Polygon", "coordinates": [ [114.204017, 4.525874],
... ] },
                         "id": "BRN" }, ... ] }
```

- 💡 구조: FeatureCollection → features(국가별 Feature) → properties.name(국가 이름), geometry(경계 좌표 Polygon), id(국가 코드).
- 📘 Choropleth를 만들 때 Folium은 해당 지역의 지리 데이터가 담긴 GeoJSON 파일이 필요.

슬라이드 16~17 코드: 캐나다 이민 세계 지도

```
        # 기본 세계 지도 (슬라이드는 Mapbox Bright 타일 사용)
        world_map = folium.Map(zoom_start=2, tiles='Mapbox Bright')

        # geojson 파일
        world_geo = r'world_countries.json'

        # 1980~2013 캐나다 이민 총합으로 choropleth 생성
        world_map.choropleth(
            geo_data=world_geo,
            data=df_canada,
            columns=['Country', 'Total'],
            key_on='feature.properties.name',
            fill_color='YlOrRd',
            legend_name='Immigration to Canada')
        world_map
```

- 📘 Recap: Mapbox Bright 타일은 지도에 모든 나라 이름을 표시.

A-5. Recap 요약 (슬라이드 18)

- Folium = 지리공간 데이터 시각화 라이브러리
- tiles 파라미터로 지도 스타일 변경 (street level, stamen 등)
- location = 지도 중심의 위도·경도
- popup = 클릭 시 라벨
- Choropleth = 통계 변수 값에 비례해 음영 처리 / GeoJSON 필수
- Mapbox Bright 타일 = 모든 나라 이름 표시

PART B. Colab 실습 (노트북 `DV_09_Creating_map_by_folium_1.ipynb`)

B-0. 전체 순서

```
[준비]     import numpy, pandas → import folium
[기본 지도] folium.Map() → location, zoom_start → tiles(스타일)
[마커]     경찰 사건 CSV → 상위 100건 → 샌프란시스코 지도 → FeatureGroup+CircleMarker → Marker(popup) → CircleMarker(popup) → MarkerCluster
[Choropleth] 이민 CSV + GeoJSON → 세계 지도 → folium.Choropleth(geo_data, data, columns, key_on, fill_color, ...)
```

- 💻 Folium을 쓰는 이유: 지리공간 시각화 전용으로 개발. plotly 같은 다른 라이브러리는 API 호출 횟수 제한이 있을 수 있지만 Folium은 완전 무료.
- 💻 Folium = Leaflet 지도를 만드는 라이브러리. 결과가 인터랙티브라 대시보드 제작에 유용. 파이썬에서 데이터를 가공하고 → Leaflet 지도에 시각화(Folium 공식 문서). GeoJSON/TopoJSON 오버레이와 데이터 바인딩으로 choropleth(color-brewer 색 구성) 지원.

B-1. 준비 (셀 5~8)

```python
import numpy as np
import pandas as pd
#!pip3 install folium==0.5.0      # 기본 설치되어 있지 않으면 설치 (주석 처리됨)
import folium
print('Folium installed and imported!')
```

B-2. 기본 지도와 확대 수준 (셀 11~21)

```python
world_map = folium.Map()                                          # 기본 세계 지도
world_map

world_map = folium.Map(location=[56.130, -106.35], zoom_start=4)  # 캐나다 중심, 낮은 줌
world_map = folium.Map(location=[56.130, -106.35], zoom_start=8)  # 더 높은 줌 → 더 확대
```

- 📘 지도의 모든 위치는 위도(Latitude)와 경도(Longitude) 로 정의. 중심을 [0, 0] 으로 줄 수도 있음.
- 📘 줌(zoom) 수준이 높을수록 중심을 더 확대해서 보여줌.
- 📘 Folium 지도는 인터랙티브: 초기 줌과 상관없이 원하는 곳을 확대 가능.

연습: 한국 지도 (셀 19~21)

```python
latitude = 35.9078
longitude = 127.7669
korea_map = folium.Map(location=[latitude, longitude], zoom_start=4)    # 문제는 줌 4 (노트북 입력 셀은 6으로 되어 있음)
korea_map
```

- ⚠️ 문제 문장·정답 셀은 zoom 4, 코드 입력 셀 예시는 zoom 6으로 서로 다름 → 시험 문제에서 주어진 숫자를 사용.
- 💡 location은 [위도, 경도] 순서 (위도 먼저!). 한국 35.9, 127.8.

B-3. 지도 스타일(tiles) (셀 22~34)

tiles 값	특징
(기본)	OpenStreetMap 스타일
`'Cartodb dark_matter'`	고대비 흑백(B+W) 지도. 데이터 매시업, 강 굽이·해안 지역 탐색에 적합
`'Cartodb positron'`	밝고 미니멀한 디자인(흰/밝은 배경, 단순한 선). 현대적이고 시각적으로 깔끔. 확대하면 국경이 나타나고 국가명이 영어로 표시

```
        # Cartodb dark_matter
        world_map = folium.Map(location=[56.130, -106.35], zoom_start=4, tiles='Cartodb
dark_matter')
        # Cartodb positron
        world_map = folium.Map(location=[56.130, -106.35], zoom_start=4, tiles='Cartodb positron')
```

- 💡 슬라이드는 Stamen Toner / Stamen Terrain / Mapbox Bright, 노트북은 Cartodb dark_matter / Cartodb positron — 모두 `tiles=` 파라미터로 이름만 바꾸는 방식.
- ➕ (슬라이드에 없는 보충) 최신 Folium에서는 Stamen·Mapbox 타일이 더 이상 기본 지원되지 않아 슬라이드 코드를 그대로 실행하면 안 나올 수 있음. 시험에서는 슬라이드/노트북 스타일명과 `tiles=` 문법만 알면 됨.

연습: 멕시코 지도 (밝고 현대적, 줌 6)

```
        mexico_latitude = 23.6345
        mexico_longitude = -102.5528
        mexico_map = folium.Map(location=[mexico_latitude, mexico_longitude], zoom_start=6,
tiles='Cartodb positron')
        mexico_map
```

B-4. 마커가 있는 지도: 샌프란시스코 경찰 사건 데이터

데이터 (셀 36~47)

- 💻 San Francisco Police Department Incidents (2016) — SFPD 범죄 신고 시스템 데이터. 주소·위치는 익명화(블록 중간 또는 교차로로 이동).

```
        df_incidents = pd.read_csv('DV_09_data_police_2016.csv')
        df_incidents.head(10)
        df_incidents.shape          # (38571, 13)
```

13개 열(features) | # | 열 | 의미 | |---|---|---| | 1 | IncidntNum | 사건 번호 | | 2 | Category | 범죄/사건 범주 | | 3 | Descript | 사건 설명 | | 4 | DayOfWeek | 요일 | | 5 | Date | 날짜 | | 6 | Time | 시간 | | 7 | PdDistrict | 경찰서 관할 구역 | | 8 |

Resolution | 처리 결과(체포 여부) | | 9 | Address | 사건 발생 근처 주소 | | 10 | X | 경도(longitude) | | 11 | Y | 위도(latitude) | | 12 | Location | (위도, 경도) 튜플 | | 13 | PdId | 경찰서 ID |

- ⚠ X = 경도, Y = 위도 (헷갈리기 쉬움!). Folium은 `[위도, 경도]` = `[Y, X]` 순서로 넣어야 함.
- 📘 노트북 본문은 "150,500건"이라 적혀 있지만 실제 `shape`은 (38571, 13). 연산량을 줄이려고 처음 100건만 사용:

```
        limit = 100
        df_incidents = df_incidents.iloc[0:limit, :]     # 행 0~99, 모든 열
        df_incidents.shape                                # (100, 13)
```

지도 만들기 (셀 48~50)

```
        latitude = 37.77
        longitude = -122.42
        sanfran_map = folium.Map(location=[latitude, longitude], zoom_start=12)   # 기본 스타일, 줌
12
```

방법 ①: FeatureGroup + CircleMarker로 위치 표시 (셀 51~52)

- 📘 Folium에서 지도에 데이터 위치를 겹치려면 feature group을 만들어 자체 feature/스타일을 부여한 뒤 지도에 추가.

```
        incidents = folium.map.FeatureGroup()                 # 피처 그룹 생성

        for lat, lng, in zip(df_incidents.Y, df_incidents.X):  # 위도(Y), 경도(X)를 짝지어 순회
            incidents.add_child(
                folium.vector_layers.CircleMarker(
                    [lat, lng],
                    radius=5,              # 원 크기
                    color='yellow',        # 테두리색
                    fill=True,
                    fill_color='blue',     # 채움색
                    fill_opacity=0.6))     # 채움 투명도

        sanfran_map.add_child(incidents)                      # 그룹을 지도에 추가
```

- 💡 `zip(a, b)` = 두 리스트를 같은 순번끼리 쌍으로 묶어 순회.

방법 ②: 팝업 텍스트 — `Marker(popup=)` (셀 53~54)

- 📘 마커 위에 마우스를 올리면(노트북 표현)/클릭하면 범죄 범주(Category) 가 표시되도록.

```
        latitudes  = list(df_incidents.Y)
        longitudes = list(df_incidents.X)
        labels     = list(df_incidents.Category)

        for lat, lng, label in zip(latitudes, longitudes, labels):
            folium.Marker([lat, lng], popup=label).add_to(sanfran_map)
        sanfran_map.add_child(incidents)
```

- 문제: 마커가 너무 많아 지도가 혼잡(congested). 해결책 2가지 ↓

해결책 A (간단): 위치 마커 제거 후 **원형 마커 자체에 팝업** (셀 55~56)

```
        sanfran_map = folium.Map(location=[latitude, longitude], zoom_start=12)
        for lat, lng, label in zip(df_incidents.Y, df_incidents.X, df_incidents.Category):
            folium.vector_layers.CircleMarker(
                [lat, lng],
                radius=5, color='yellow', fill=True,
                popup=label,                                    # 원형 마커에 팝업
                fill_color='blue', fill_opacity=0.6
            ).add_to(sanfran_map)                               # 지도에 직접 추가
```

해결책 B (정석): **MarkerCluster로 묶기** (셀 57~59) ⭐

- 📘 마커를 군집(cluster) 으로 묶어, 각 클러스터에 그 지역의 범죄 수를 표시. 클러스터는 샌프란시스코의 지역별 묶음(pocket) 으로 따로 분석 가능.

```
        from folium import plugins

        sanfran_map = folium.Map(location=[latitude, longitude], zoom_start=12)

        incidents = plugins.MarkerCluster().add_to(sanfran_map)      # MarkerCluster 객체를 지도에 추가

        for lat, lng, label in zip(df_incidents.Y, df_incidents.X, df_incidents.Category):
            folium.Marker(location=[lat, lng], icon=None, popup=label).add_to(incidents)   # 지도가 아니라 cluster에 추가
        sanfran_map
```

- 📘 해석: 최대로 축소하면 모든 마커가 하나의 "전역 클러스터(global cluster)" = 100개(전체 범죄 수) 로 합쳐짐. 확대할수록 작은 클러스터로 쪼개지고, 끝까지 확대하면 개별 마커가 됨.
- 💡 핵심 차이: 일반 마커는 `.add_to(지도)`, 클러스터 마커는 **`.add_to(cluster)`**.

B-5. Choropleth Map: 세계 이민 지도

개념 (셀 60~61)

- 📘 Choropleth = 통계 변수의 측정값에 비례해 지역을 음영/무늬로 표시한 주제도. 인구밀도, 1인당 소득 같은 값이 지역별로 어떻게 달라지는지, 한 지역 내 변동 수준을 쉽게 보여줌. (예: 주별 제곱마일당 인구 지도)

데이터 (셀 62~67)

```
        df_can = pd.read_csv('DV_05_data_canada_immigration.csv')
        print(df_can.shape)        # (195, 40)
```

GeoJSON + 기본 지도 (셀 68~69)

- 📘 Choropleth에는 관심 지역(주/카운티/국가)의 경계를 정의한 GeoJSON 파일이 필요. 세계 지도 → 모든 나라 경계가 있는 GeoJSON.

```
        world_geo = r'DV_09_json_world_countries.json'                # GeoJSON 파일 경로 (r'' = raw string)
        world_map = folium.Map(location=[0, 0], zoom_start=2)         # [0, 0] 중심, 초기 줌 2
```

`choropleth` 주요 파라미터 (셀 70~71) ⭐ 시험 단골

파라미터	의미
`geo_data`	GeoJSON 파일(경계 데이터)
`data`	데이터가 들어 있는 DataFrame
`columns`	choropleth에 쓸 DataFrame의 열 — `['Country', 'Total']` ([키 열, 값 열])
`key_on`	GeoJSON 안에서 관심 변수(국가 이름)를 담은 키. GeoJSON을 텍스트 편집기로 열어 국가 이름이 들어 있는 키를 확인해야 함. 여기서는 `name`. 대소문자 구분 — GeoJSON에 있는 그대로 `'feature.properties.name'`

```
        world_map = folium.Map(location=[56.1304, -106.3468], zoom_start=2)

        folium.Choropleth(
            geo_data=world_geo,                     # 경계 GeoJSON 경로/객체
            data=df_can,                            # 국가별 이민 데이터가 든 DataFrame
            columns=['Country', 'Total'],           # 사용할 열
            key_on='feature.properties.name',       # DataFrame의 Country와 GeoJSON의 국가 이름을 연결
            fill_color='YlOrRd',                    # 색 스케일 (노랑-주황-빨강)
            fill_opacity=0.7,                       # 채움 투명도
            line_opacity=0.2,                       # 경계선 투명도
            nan_fill_color='lightgray',             # 결측값(NaN) 국가 색
            nan_fill_opacity=0.4,                   # 결측값 국가 투명도
            legend_name='Immigration to Canada',    # 범례 제목
            reset=True                              # 다시 불러올 때 리셋
        ).add_to(world_map)
        world_map
```

- 📘 해석: 범례상 색이 진하고 빨간색에 가까울수록 이민자가 많음. 1980~2013년 가장 많은 이민: China, India, Philippines, 이어서 Poland, Pakistan, 그리고 (흥미롭게도) US.
- 💡 연결 원리: `data`의 `Country` 값 ↔ GeoJSON의 `feature.properties.name` 값이 글자 그대로 일치해야 그 나라가 칠해짐. 일치하지 않으면 NaN → **`nan_fill_color`**(회색).
- 💡 슬라이드는 `world_map.choropleth(...)`(구 방식), 노트북은 `folium.Choropleth(...).add_to(world_map)`(신 방식). 파라미터 이름은 동일(`geo_data, data, columns, key_on, fill_color, legend_name ...`).
- 📘 확장 아이디어: 개별 연도나 10년 단위로 choropleth를 만들어 전체 기간(1980~2013)과 비교.

PART C. 정리표

C-1. Folium 객체·함수 정리

하고 싶은 것	코드	핵심 파라미터
지도 만들기	`folium.Map(...)`	`location=[위도, 경도]`, `zoom_start`(클수록 확대), `tiles`(스타일)
지도 스타일	`tiles='Cartodb dark_matter'`, `'Cartodb positron'` (슬라이드: `'Stamen Toner'`, `'Stamen Terrain'`, `'Mapbox Bright'`)	
마커(핀)	`folium.Marker(location, popup=).add_to(map)`	`popup`(클릭 라벨)
원형 마커	`folium.vector_layers.CircleMarker([lat,lng], radius, color, fill, fill_color, fill_opacity, popup)` (슬라이드: `folium.features.CircleMarker`)	
마커 묶음 레이어	`folium.map.FeatureGroup()` + `.add_child(...)` → `map.add_child(group)`	
군집	`plugins.MarkerCluster().add_to(map)` → 마커를 `.add_to(cluster)`	`from folium import plugins` / `from folium.plugins import MarkerCluster`
Choropleth	`folium.Choropleth(geo_data, data, columns, key_on, fill_color, fill_opacity, line_opacity, nan_fill_color, nan_fill_opacity, legend_name, reset).add_to(map)`	GeoJSON 필수

C-2. plot별 사용 변수(데이터)

지도	데이터	위치 정보	값/라벨
기본 지도	없음	`location=[위도, 경도]`	—
마커 지도	`df_incidents`	Y(위도), X(경도)	`Category`(popup)
클러스터 지도	동일	Y, X	`Category`
Choropleth	`df_can` + GeoJSON	Country ↔ GeoJSON name (`key_on`)	**`Total`** (색 농도)

C-3. 시험 대비 포인트

1. Folium = 지리공간 시각화 / 위도·경도 / 마커·클러스터 / 다양한 스타일 / 무료(plotly는 API 호출 제한 가능).
2. `folium.Map(location=[위도, 경도], zoom_start=N, tiles='스타일')` — location은 [위도, 경도] 순서, zoom_start ↑ = 확대.

3. tiles로 스타일 변경 (Cartodb dark_matter = 흑백 고대비, Cartodb positron = 밝고 미니멀, Stamen Toner/Terrain, Mapbox Bright = 모든 나라 이름 표시).
4. popup = 클릭하면 나타나는 라벨. 마커 많으면 → ① CircleMarker에 popup ② MarkerCluster.
5. MarkerCluster: 전체 축소 시 전역 클러스터(전체 개수, 예: 100), 확대하면 분리 → 개별 마커. 마커를 cluster에 add.
6. 데이터프레임 X = 경도, Y = 위도 → 지도에는 `[Y, X]`.
7. Choropleth = 값에 비례해 지역 색칠. GeoJSON 필수. 파라미터 `geo_data / data / columns / key_on / fill_color / legend_name`. **`key_on='feature.properties.name'`** 은 대소문자 구분, GeoJSON 키 이름 확인 필요.
8. 색이 진한 빨강일수록 이민자 많음(YlOrRd). 상위: China, India, Philippines.
9. GeoJSON 구조: FeatureCollection → features → (properties.name, geometry.Polygon, id).